Technische Universität Dresden

Dr. Thomas Springer
Fakultät Informatik | Chair of Distributed and Networked Systems

Distributed Systems
Architectural Patterns for Distributed Systems

winter term 2026/2027

Learning Objectives of this Lecture

At the end of the lecture, you can:

- **design** architectures based on system descriptions and requirements by **selecting** appropriate approaches and **applying** your knowledge of architectural patterns for distributed systems.

- **analyze** given distributed systems to **identify** their architecture and potential problems.

Distributed Software Architecture

Role of architectural design

Sommerville, I., Software Engineering (9th. ed.). Addison-Wesley Publishing Company, USA., ISBN:978-0-13-703515-1, 2010

System architecture established at an early stage of the (agile) development process

Critical link between requirements engineering and software design

- ➢Fulfillment of non-functional requirements heavily depends on the system architecture

Incremental development of architectures is usually not successful since refactoring a system architecture is likely to be expensive

An architectural model identifies the main structural components and their interrelationships

Programming in the large vs. Programming in the small

We argue that structuring a large collection of modules to form a "system" is an essentially distinct and different intellectual activity from that of constructing the individual modules. That is, we distinguish programming-in-the-large from programming-in-the-small. Correspondingly, we believe that essentially distinct and different languages should be used for the two activities.

Frank DeRemer and Hans Kron. 1975. Programming-in-the large versus programming-in-the-small. SIGPLAN Not. 10, 6 (June 1975), 114–121. https://doi.org/10.1145/390016.808431

Programming in the large (handling the forrest)

- knitting the modules together into an integrated whole with a „module interconnection language“

Programming in the small (handling the trees)

- Implementation of the individual modules with a typical programming language

Architectural design decisions

Architectural design is a **creative process**

Sommerville, I., Software Engineering (9th. ed.). Addison-Wesley Publishing Company, USA., ISBN:978-0-13-703515-1, 2010

- **Series of structural decisions** dependent on the type and specific requirements of the system to develop
- **Principle of architectural decomposition** - components with sub-components at multiple levels of hierarchy
- Usually based on an **architectural pattern**

- Close relationship between non-functional properties and software architecture
- Usually search for trade-offs between competing or even conflicting non-functional requirements
  - Performance vs. security
  - Performance vs. maintainability (large, coarse-grained vs. small, fine-grained components)

Coupling: Degree of Dependence

Coupling *refers to the degree of interconnectedness or dependency between different components, modules, or classes within a system. It measures how much one part of the software relies on another. Coupling is a critical concept, as it impacts the system's maintainability, reusability, scalability, and overall flexibility.*

Edward Yourdon and Larry L. Constantine. Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design (1st. ed.). Prentice-Hall, Inc., USA. 1979.

Simple Counting of connections is usually insufficient

Measure of the strength of intermodular interconnection

- Two modules are totally independent if each can function completely without the presence of the other.
- Strong connections – highly coupled
- Week connections – loosely coupled

Dimensions of Coupling

Complexity of the interface	Number of interfaces, operations per interface, number of parameters
Type of information flow along the connection	Data, control or both
Binding time of the connection	Code, compilation time, run-time

Cohesion: Functional Relatedness

Cohesion *refers to the degree to which elements within a module or component of a system are related and work together to perform a specific function or set of related functions. High cohesion is a desirable attribute as it indicates that the elements within a module are closely related and focused on a specific purpose, making the module easier to understand, use, and maintain.*

Edward Yourdon and Larry L. Constantine. Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design (1st. ed.). Prentice-Hall, Inc., USA. 1979.

Measure of **intramodular relatedness**

- How tightly related or bound are the internal elements of a module?

Levels of cohesion (order of increasing strength)

Coincidental association	Little or no constructive relationship among the elements of a module
Logical association	Similar or related functions (e.g., manage documents, operations on customer)
Temporal	Elements related by time, e.g. used or executed at the same time (e.g., invocation order)
Procedural	Elements represent steps in a specific algorithm or procedure
Communicational	Elements operate on the same input data or share intermediate results
Sequential	Elements need to be executed in a particular order (e.g., output data of one element is input of another)
Functional	Elements contribute to a single, well-defined task or function

Coupling and Cohesion in Architectural Design

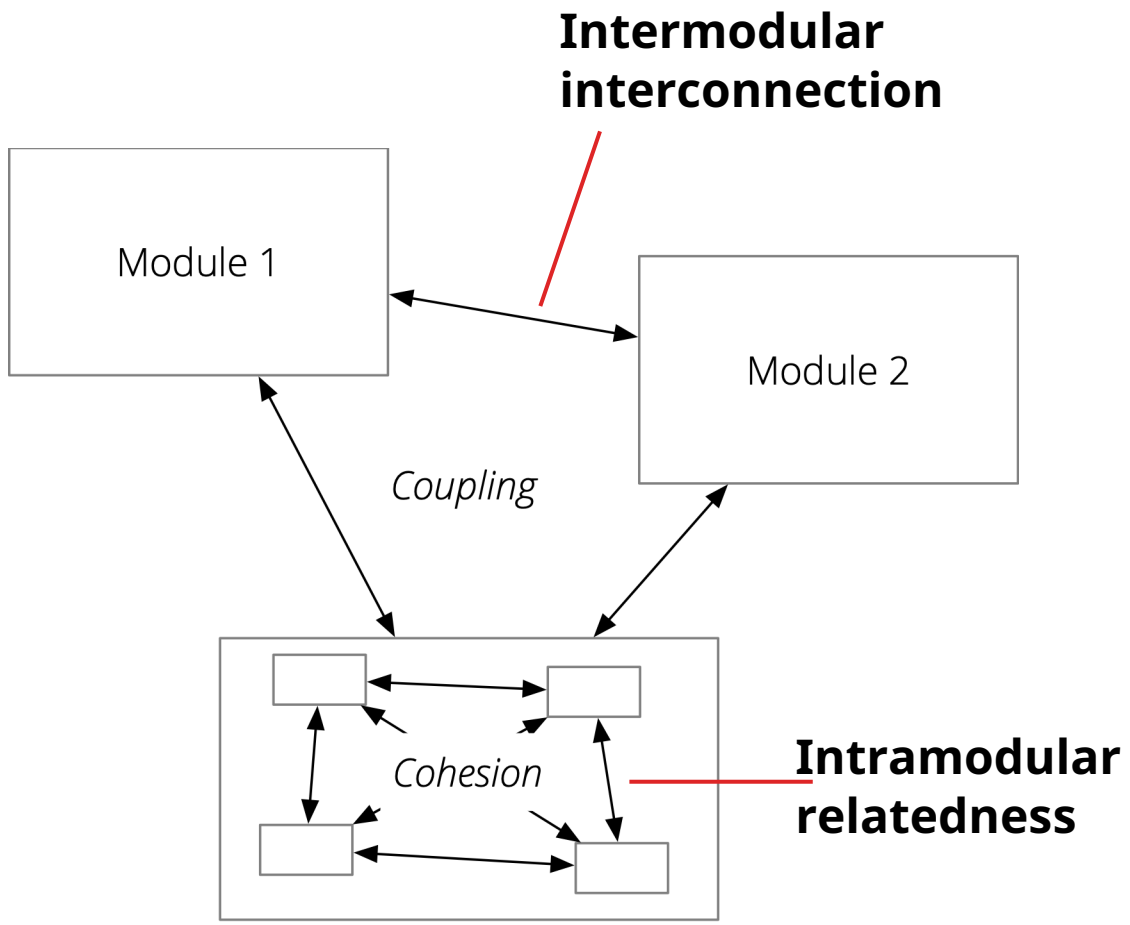
Common design approach for Distributed Systems:

➢ loose coupling and high cohesion

Separation of Concerns:

> *design principle for organizing software so that each element in the program does one thing and one thing only*

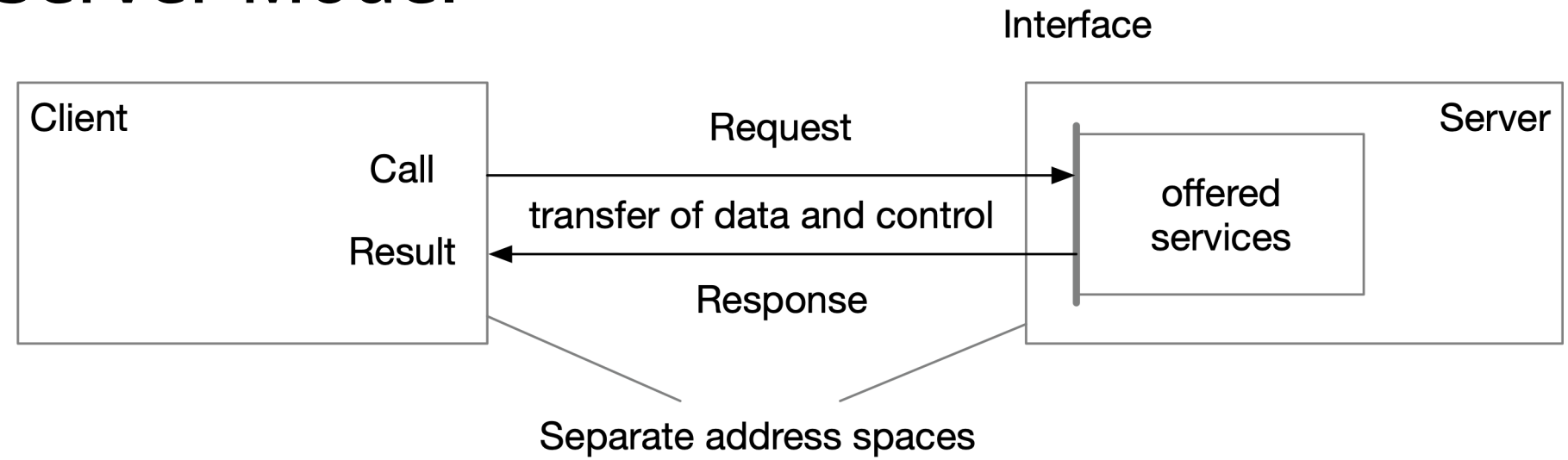
- Understand each element without the need to understand other elements
- Changes are localized to a small number of elements



Capture the essence of an architecture that has been used in different software systems

ARCHITECTURAL PATTERNS

Client/Server Model



Roles of components in service usage

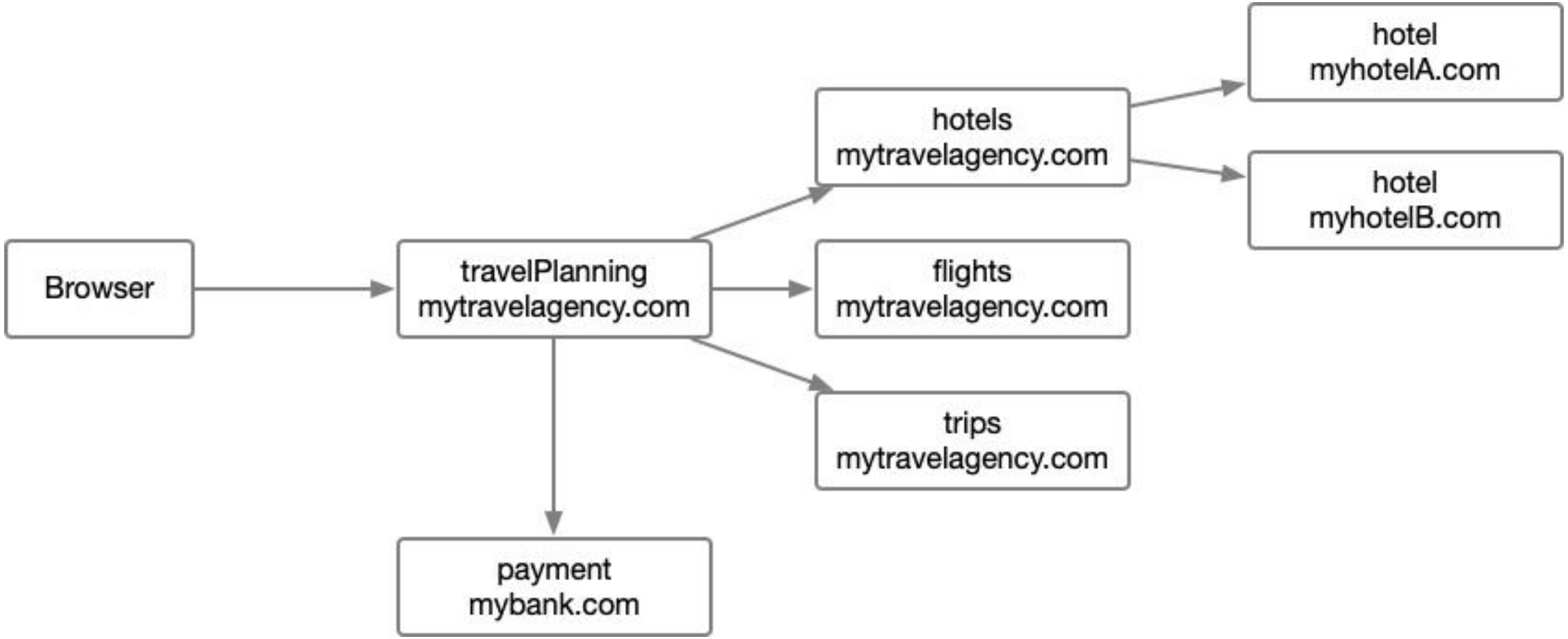
- Client component: Requesting/Invoking an operation/service
- Server component: Implementing an operation/service and providing a result

Roles of nodes (according to the main purpose of a node)

- Client node: includes interaction with user, mainly for client role - Laptop, Desktop PC, Smartphone
- Server node: no direct interaction with user, mainly for processing in server role - server machines, VMs in datacenter

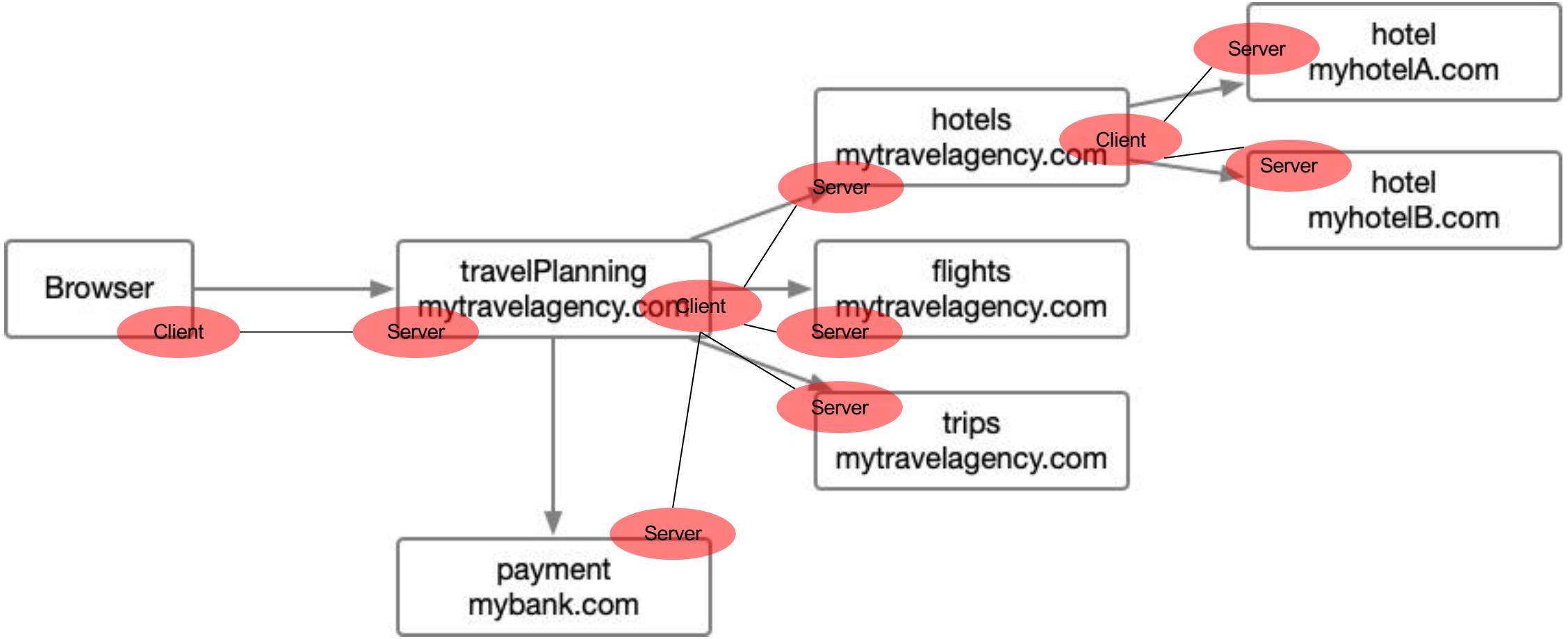
Task: Client and Server Roles for Service Usage

The arrows denote that a component requests a service from the component the arrow is pointing at. Identify the roles of all components according to their service usage.



Task: Client and Server Roles for Service Usage

The arrows denote that a component requests a service from the component the arrow is pointing at. Identify the roles of all components according to their service usage.



Layered Software Architecture

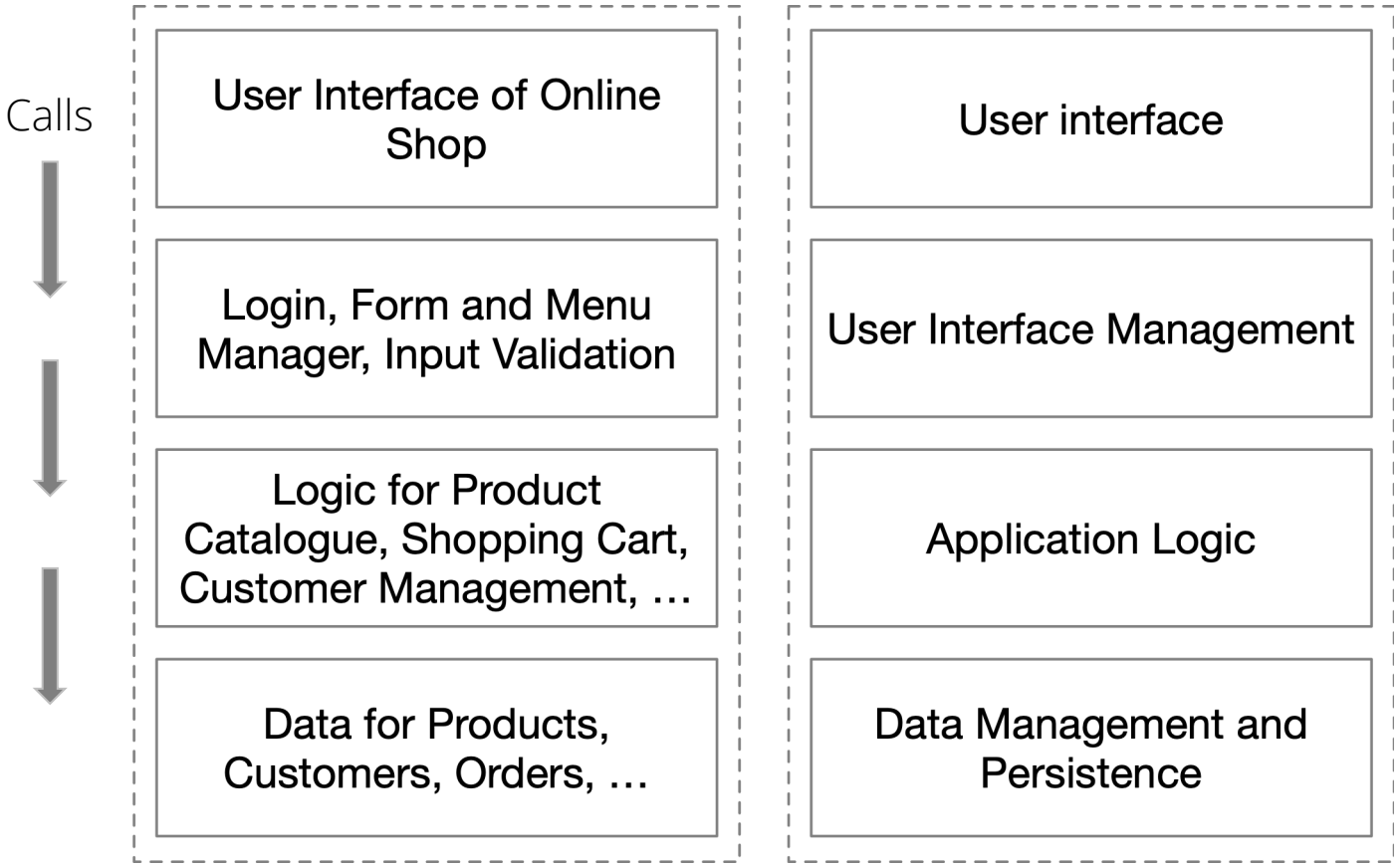
Typical structure of Information Systems

System functionality is logically organized into separate layers with well defined interfaces

Each layer only relies on the facilities and services offered by the layer immediately beneath it

Separation of layers allow independent changes and management as long as the interfaces are unchanged

Interface changes only affect the adjacent layer



How to distribute?

Vertical distribution *refers to the distribution of the different layers in a multitiered architecture across multiple machines. In principle, each layer is implemented on a different machine.*

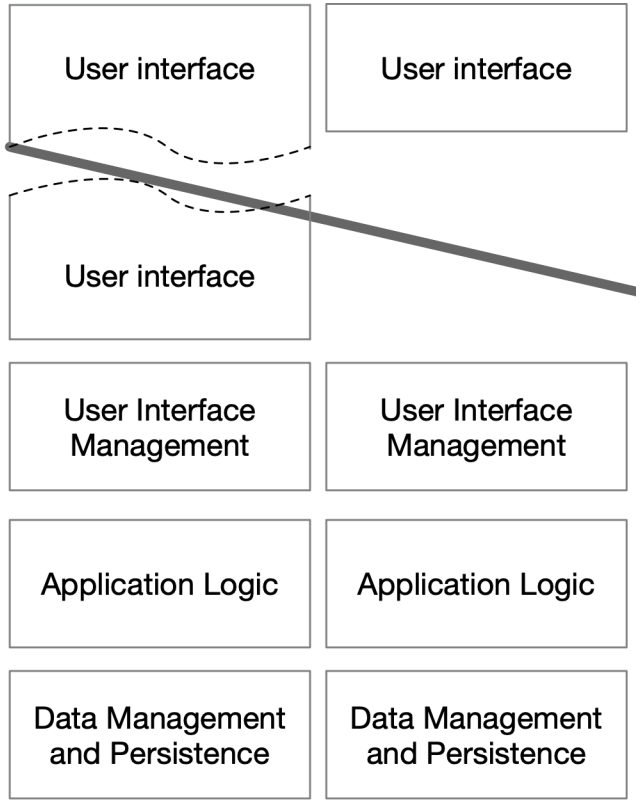
van Steen, M., Tanenbaum, A.S.: Distributed Systems – Principles and Paradigms, Pearson, 2017

Horizontal distribution *deals with the distribution of a single layer across multiple machines, such as distributing a single database.*

van Steen, M., Tanenbaum, A.S.: Distributed Systems – Principles and Paradigms, Pearson, 2017

Two-Tier Architecture

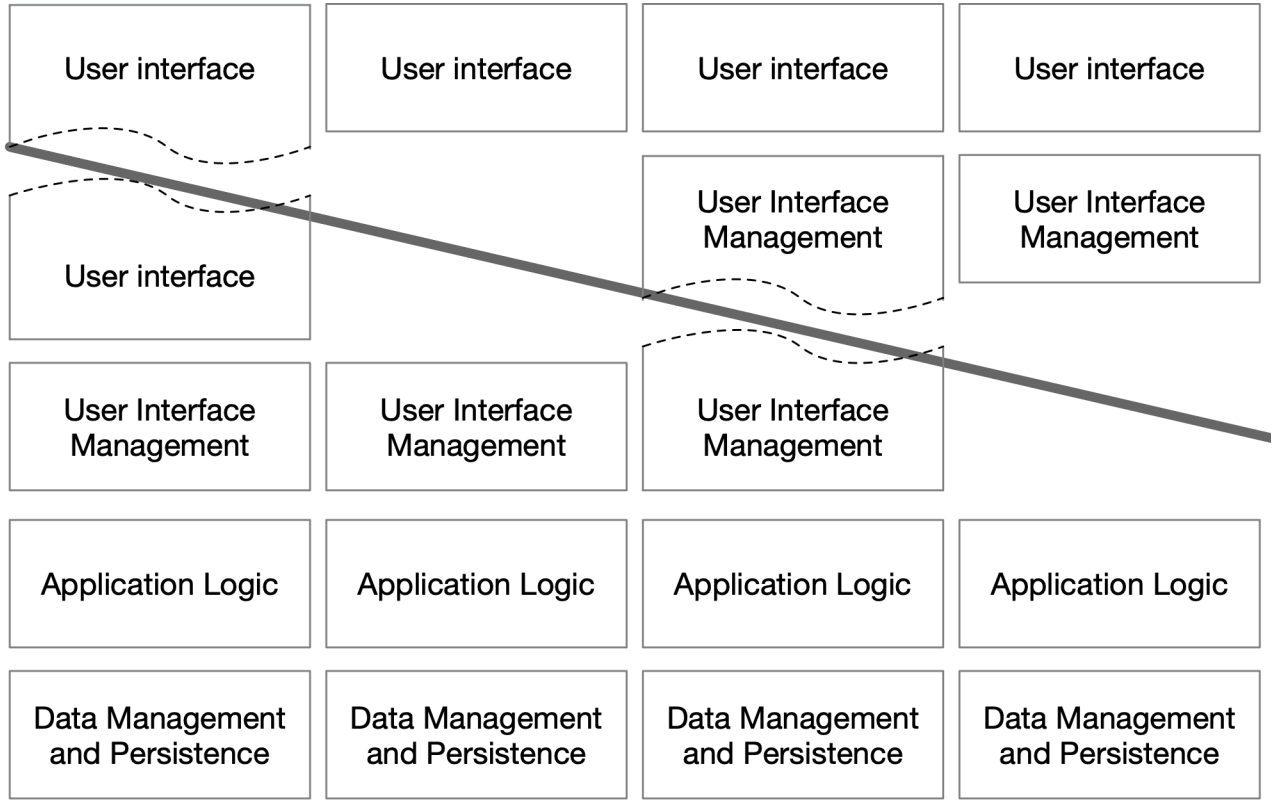
Vertical Distribution



Layers are a logical system organization without any assumption on their distribution, while ***tiers*** refer to the physical distribution of system parts concerning (physical) nodes.

Two-Tier Architecture

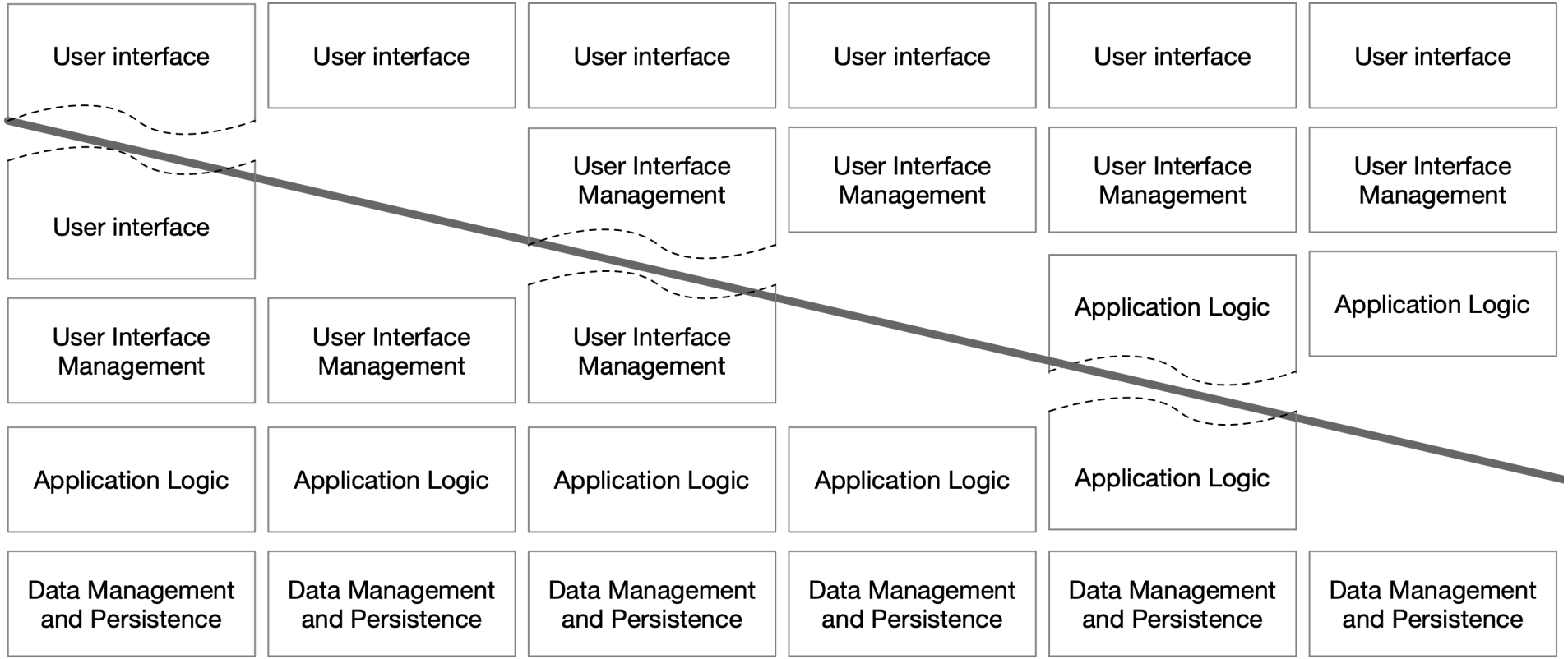
Vertical Distribution



Layers are a logical system organization without any assumption on their distribution, while ***tiers*** refer to the physical distribution of system parts concerning (physical) nodes.

Two-Tier Architecture

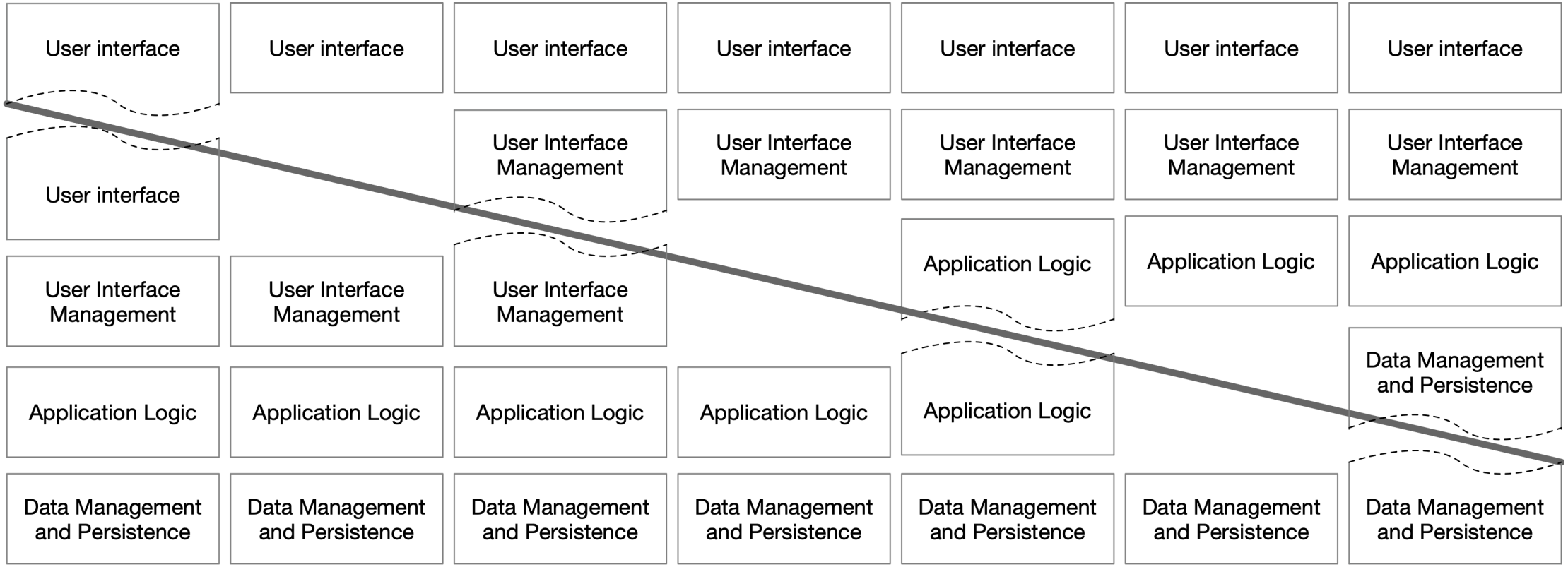
Vertical Distribution



Layers are a logical system organization without any assumption on their distribution, while ***tiers*** refer to the physical distribution of system parts concerning (physical) nodes.

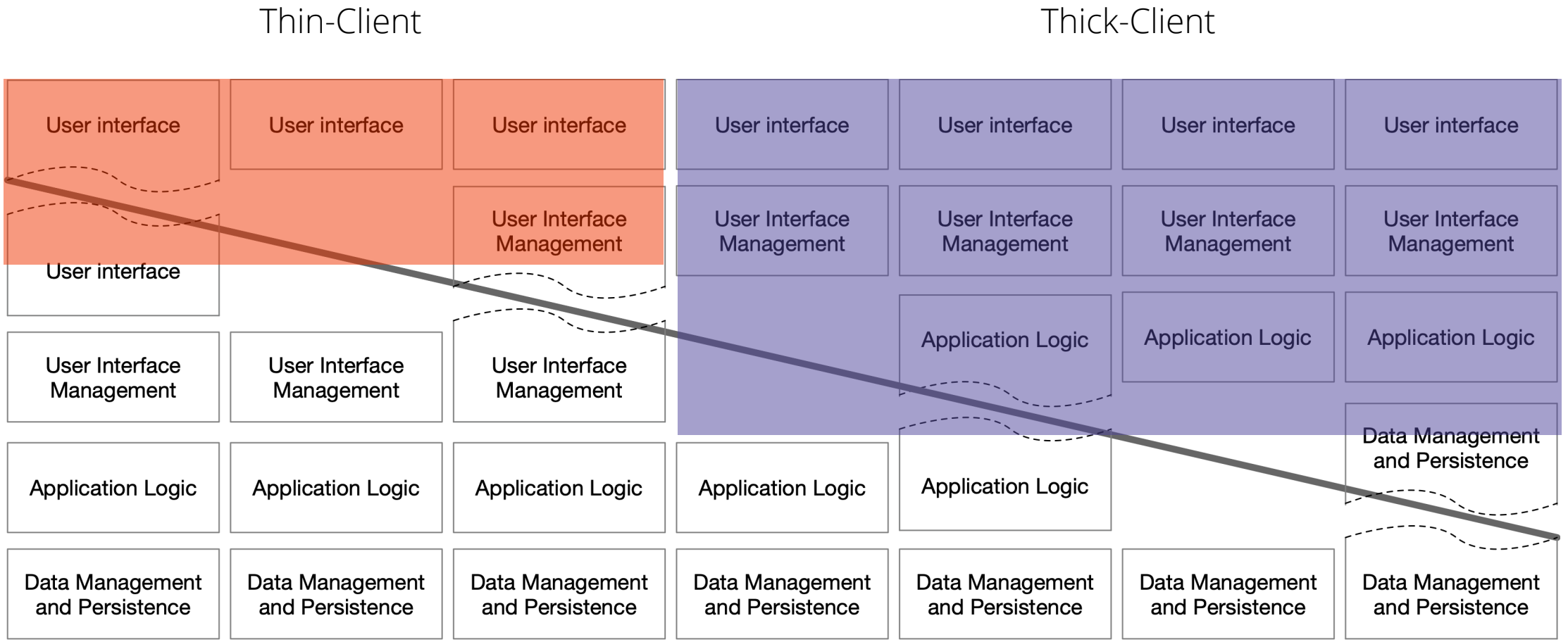
Two-Tier Architecture

Vertical Distribution



Layers are a logical system organization without any assumption on their distribution, while ***tiers*** refer to the physical distribution of system parts concerning (physical) nodes.

Where to split? - Thin- vs. Thick Client

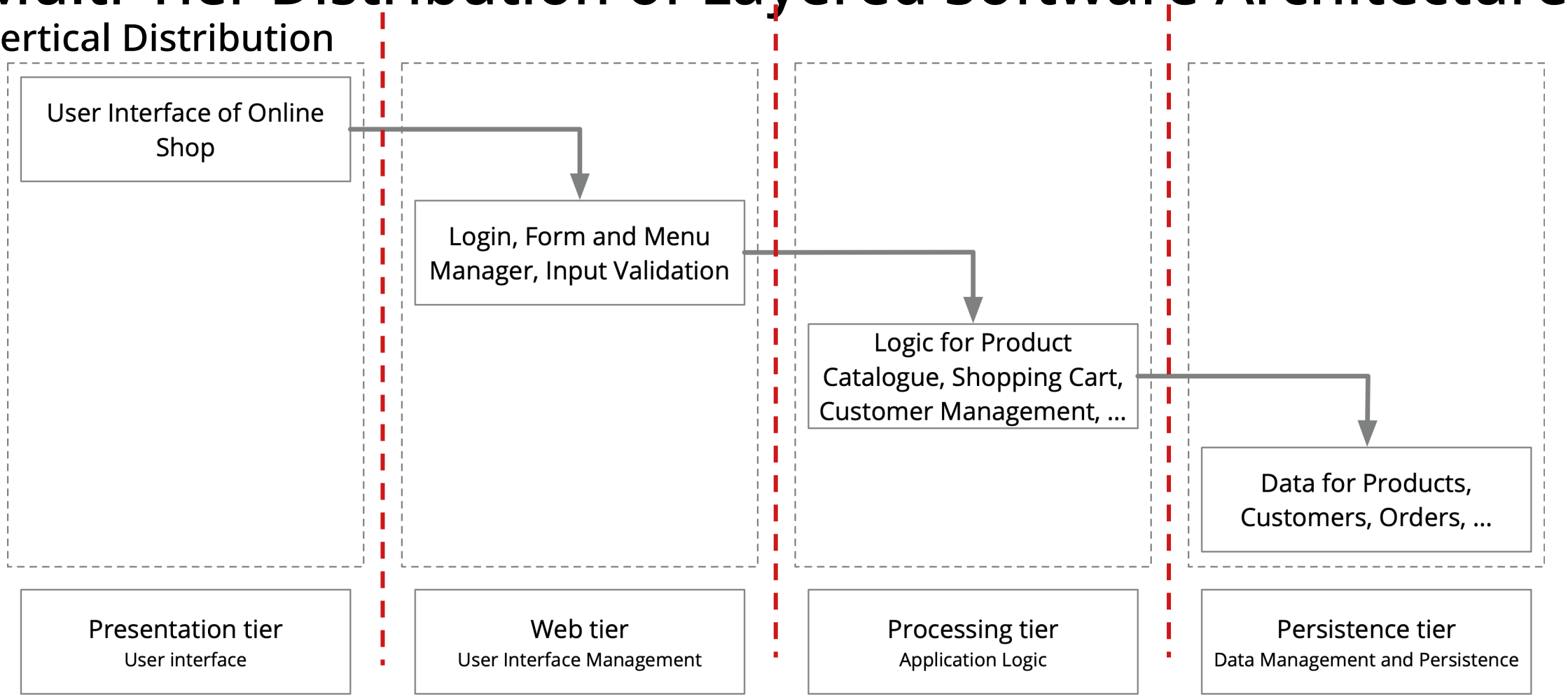


Thin client: no or minimal processing on the client, most of the processing is performed on the server-side, strong dependence of client on server

Thick client: larger amount of processing on client, client relies to server-side only from time to time, weaker dependence of client on server

Multi-Tier Distribution of Layered Software Architecture

Vertical Distribution



Clear separation of concerns and well-defined interfaces between layers allow independent configuration, management, and change within each tier.

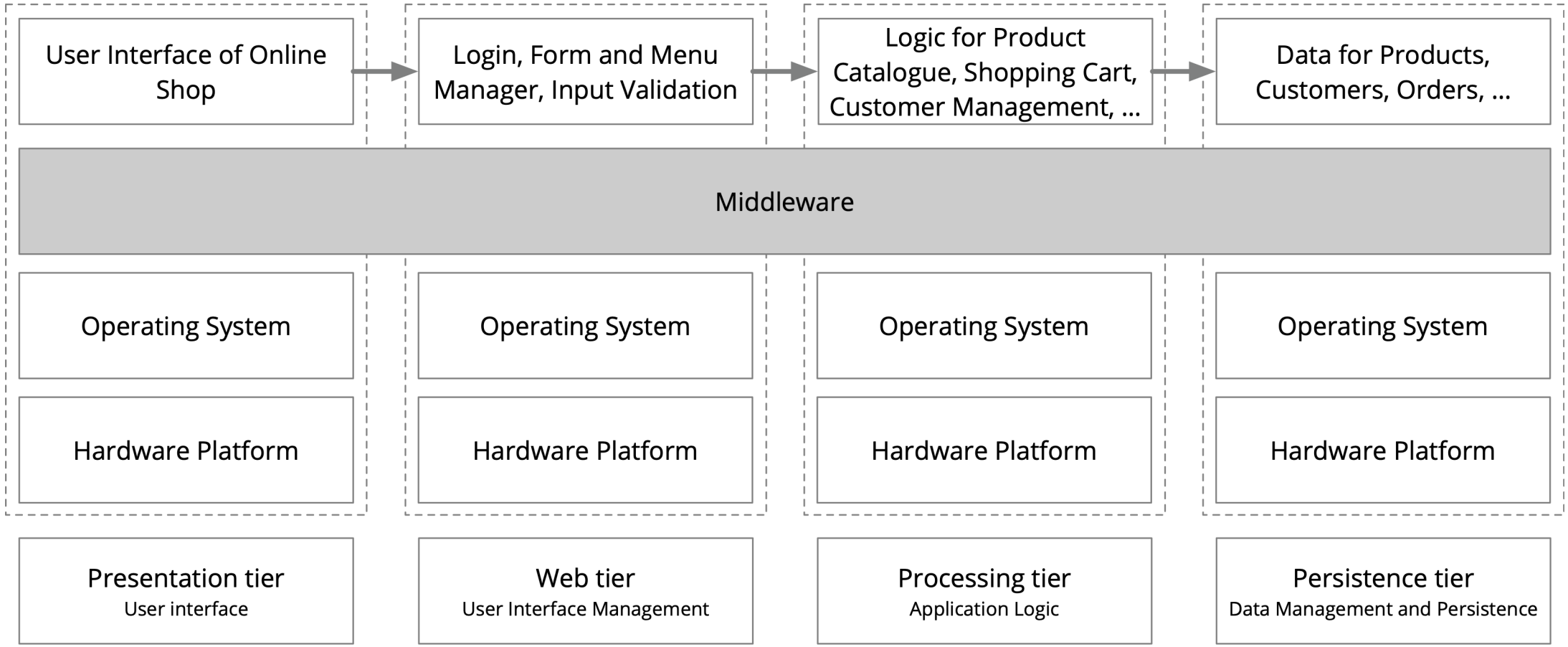
Task: Sketch a Three-Tier Architecture for …

An online shop for selling books should allow users to browse in the product catalogue so see available books with prices and description as well as ratings given by other users.

Task: Sketch a three-tier architecture of the online book shop. Use a simple "lines and boxes" notation and choose an appropriate level of detail to sketch a high level architecture of the complete system setup. Assign the identified components to a tier each in a multi-tier architecture.

Upload your sketch to the “Upload“ folder in OPAL

Middleware



Middleware is a software layer comprising a collection of reusable functionality, situated between the operating system and the application with special support for distributed computing systems

COMPONENT-BASED SOFTWARE DEVELOPMENT AND APPLICATION SERVERS

Software Components

*A **component** is a unit of composition with contractually specified interfaces and explicit context dependency only. A component can be deployed independently and is subject to composition by third parties.*

Szyperski, C.; Gruntz, D. & Murer, S.: Component Software - Beyond Object-Oriented Programming. Addison-Wesley Longman, Amsterdam, 2002

Unit – Specified, implemented and deployed only as a whole, designed for reuse

Contractual interfaces

- Use Contracts - interface provider and interface user
  - Fulfilment of underlying conditions : e.g. service quality
- Realisation Contracts - component specification and implementations
  - Agreement that the component specification must be respected by every implementation

Software Components

*A **component** is a unit of composition with contractually specified interfaces and explicit context dependency only. A component can be deployed independently and is subject to composition by third parties.*

Szyperski, C.; Gruntz, D. & Murer, S.: Component Software - Beyond Object-Oriented Programming. Addison-Wesley Longman, Amsterdam, 2002

Explicit context dependency

- No hidden dependencies on internal super-classes, special libraries, etc.
- Clear definition of component model and component platform (e.g., component lifecycle)

Deployed independently

- contains application logic that can be used as building block in different applications

Subject to composition by third parties

- designed for reuse

Software Components – Four Views

Four views on a component according to Cheeseman and Daniels:

Component Specification

- “Casing” of a component; independent of its implementation
- Specification of components’ offered and required interfaces

Component Implementation

- Implementation of a component according to a given specification
- Different implementations can belong to a particular specification

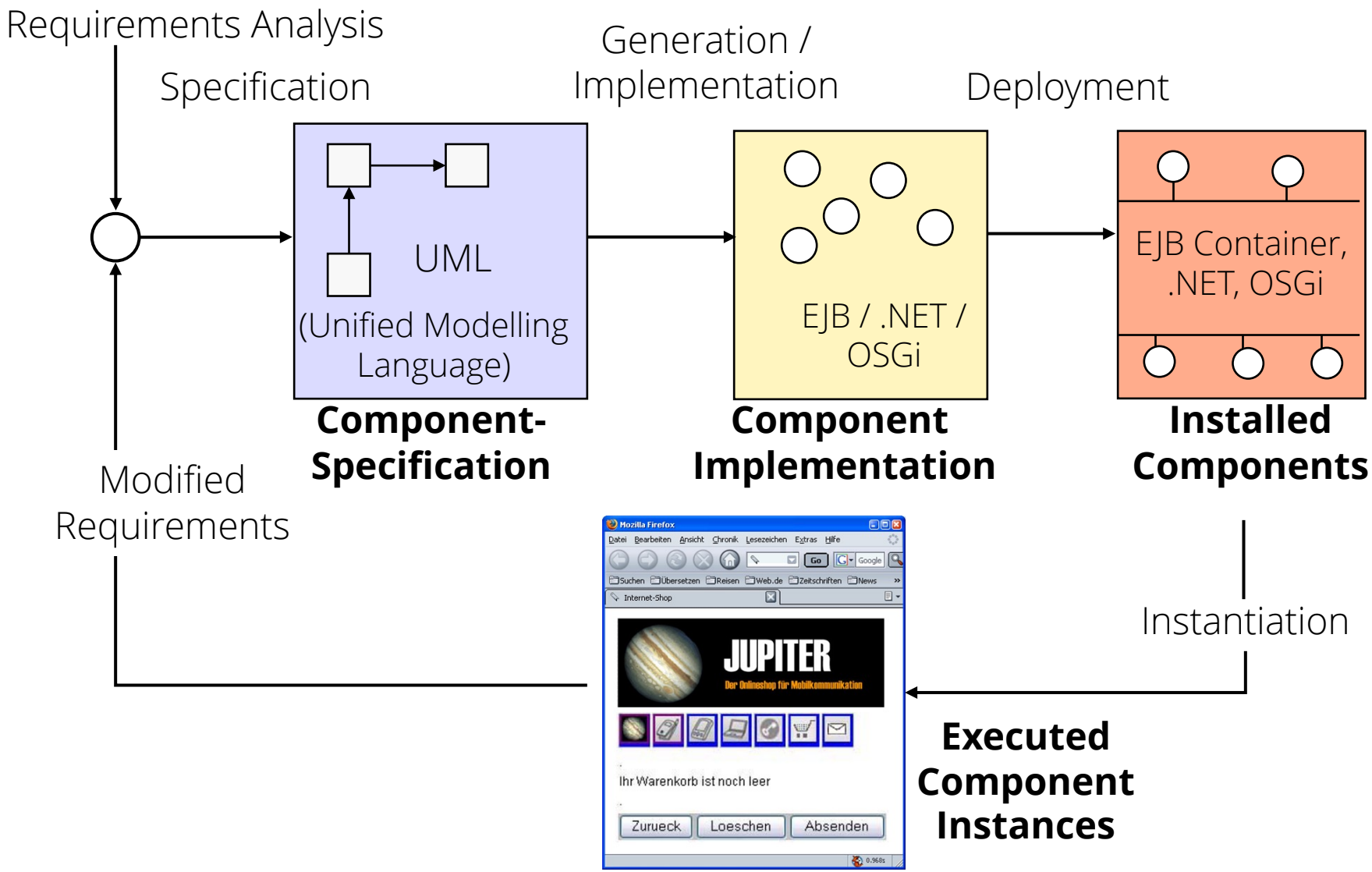
Installed Component (Deployment)

- Installation of component implementations on a component platform
- Configuration information separate from component code

Component Object (Instance)

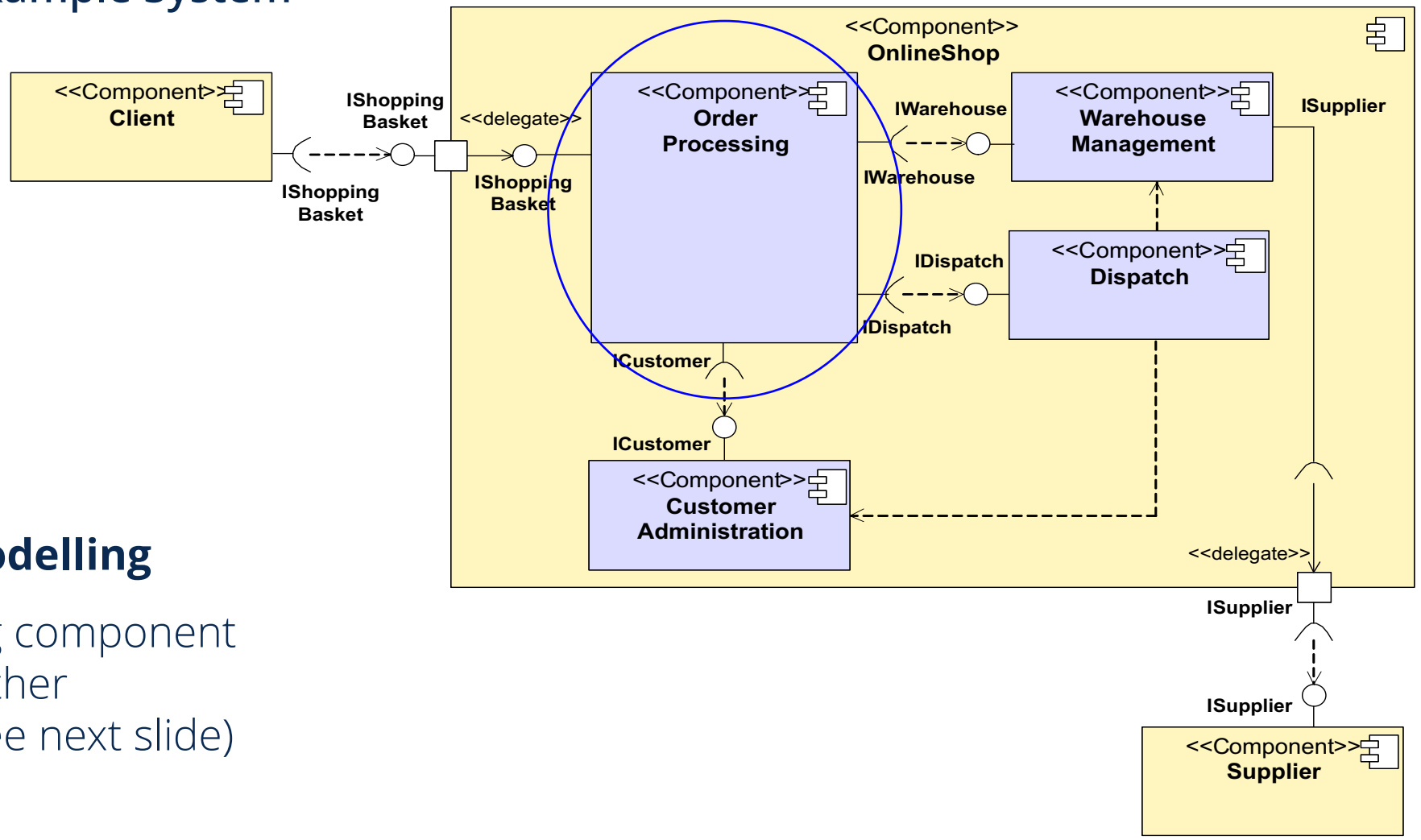
- Representation of an instance of a component in runtime with possession of an identity

Component-based Software Design



Specification of System Architecture

UML Diagram of Example System

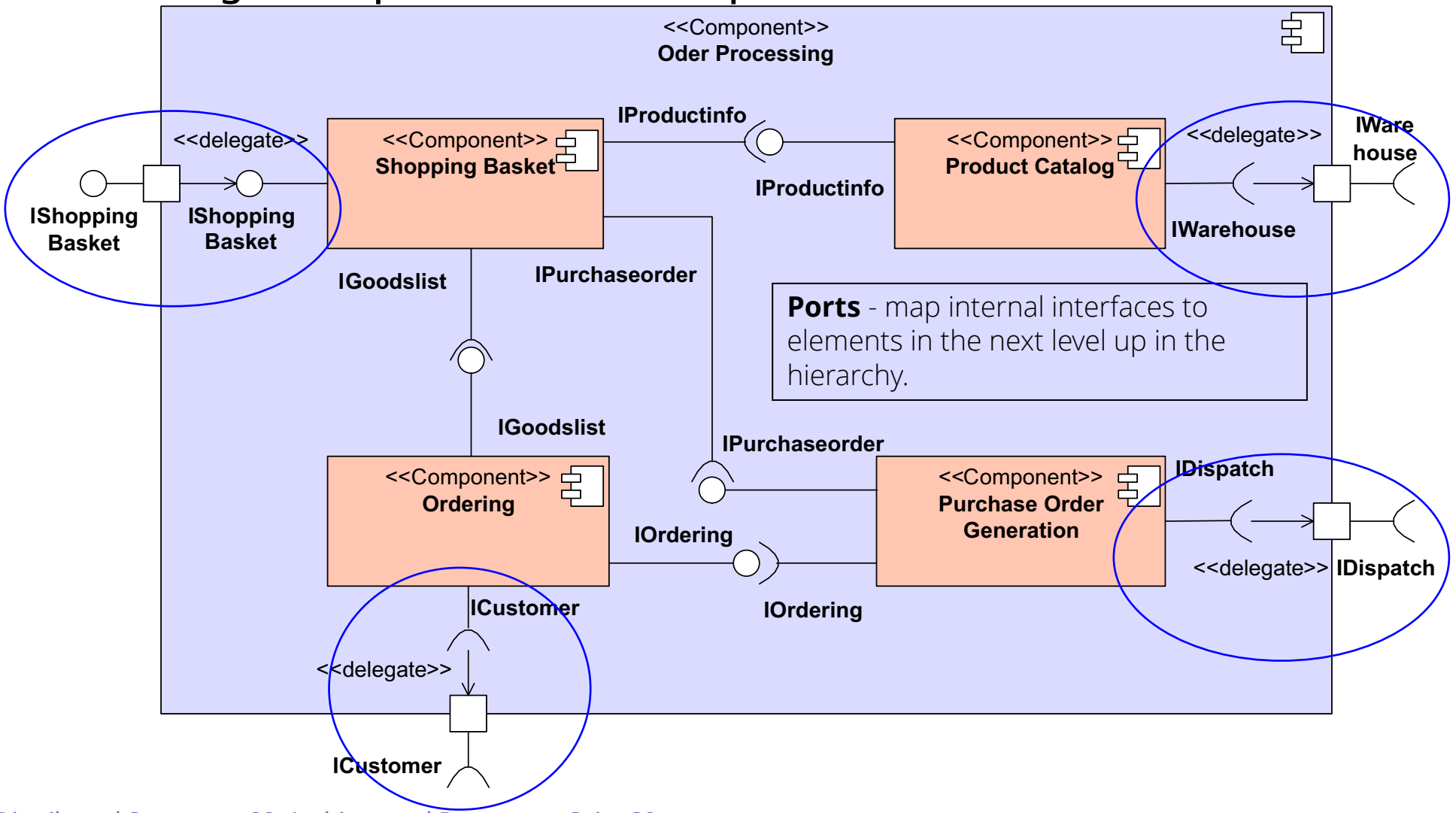


Hierarchical Modelling

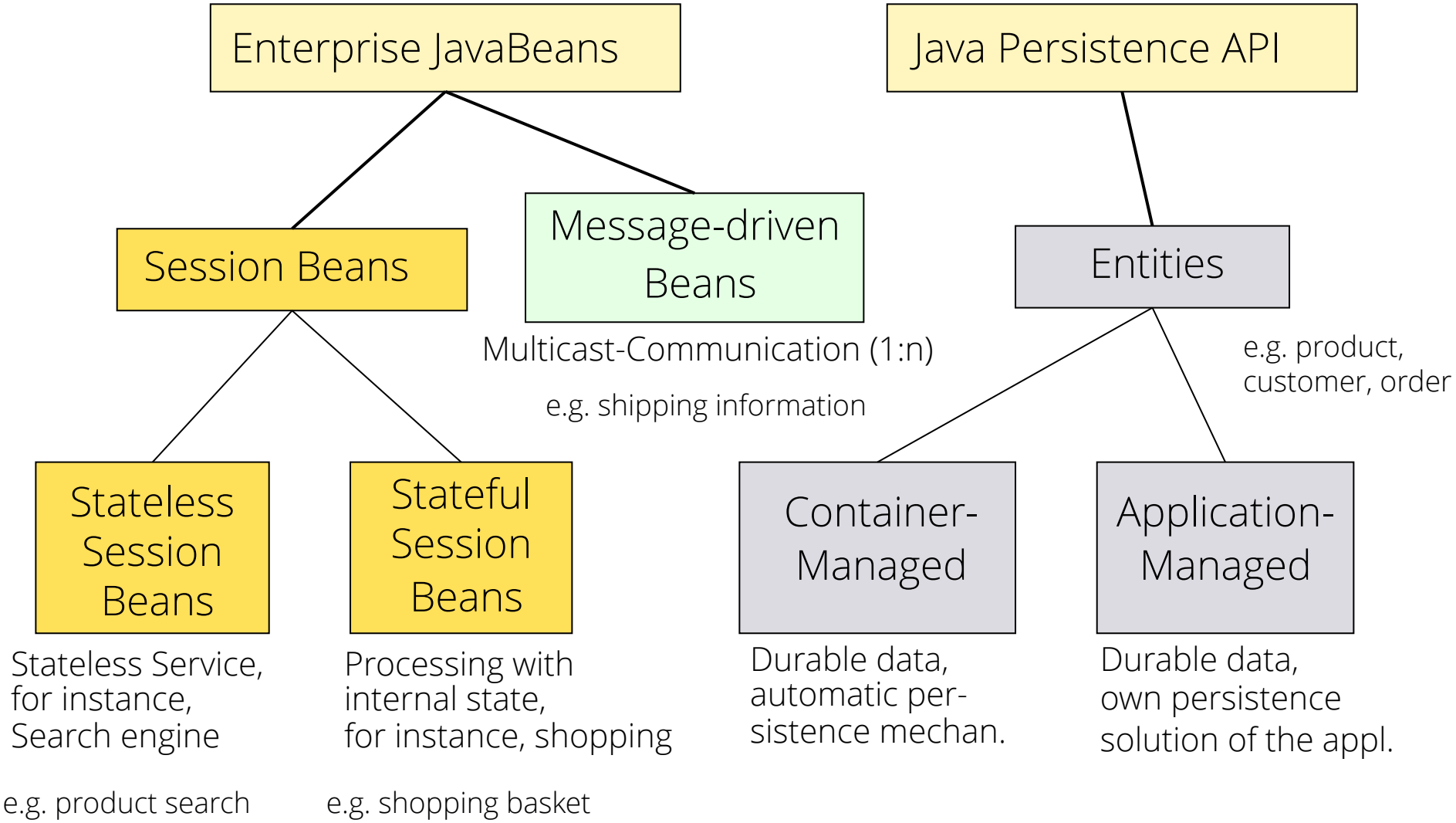
Order Processing component composed of further components. (See next slide)

Specification of System Architecture

Hierarchic Modelling – Component out of components



Component Model for Implementation and Deployment



EJB – Session Beans and Message Driven Beans

Session Bean

- Client entry point to server-side application logic for user sessions
- Type 1: Stateless
  - No information about state of session, can be deleted and then re-created later
  - E.g. Product search component (i.e., depending only on current search term)
- Type 2: Stateful
  - Information about current session with conversational state
  - E.g. Contents of shopping basket

Message Driven Bean

- Stateless
- Asynchronous communication based on messaging (timely decoupled communication)
- E.g., used for realization of business processes like purchase order processing

Entity (Java Persistence API)

- Mapping of objects to (relational) database storage
- E.g., Product Catalogue POJO(Plain Old Java Object) represents one table, rows contain instances

Explicit deployment specification

Deployment descriptor: specifies type of beans, interfaces, class implementations

Example Deployment Descriptor:

```
<ejb-jar>
 <display-name>ShoppingBasket</display-name>
  <enterprise-beans>
    <session>
     <ejb-name>ShoppingBasketBean</ejb-name>
     <remote>tud.onlinetrader.orderprocessing.ShoppingBasketRemote</remote>
     <ejb-class>tud.onlinetrader.orderprocessing.ShoppingBasketBean</ejb-class>
         <session-type>Stateful</session-type>
         <transaction-type>Container</transaction-type>
    </session>
  </enterprise-beans>
  <assembly-descriptor>  … // See next slide <assembly-descriptor>
</ejb-jar>
```

Explicit deployment specification

Assembly descriptor: specification of composition, transaction management, state management, security properties

```
<assembly-descriptor>
  <container-transaction>
    <method>
      <ejb-name> ShoppingBasketBean </ejb-name>
      <method-name>initiateOrder</method-name>
     </method>
     <trans-attribute>RequiresNew</trans-attribute>
  </container-transaction> ...

  <method-permission>
    <role-name>CorporateClient</role-name>
    <method>
      <ejb-name>ShoppingBasketBean</ejb-name>
      <method-name>corporateOrder</method-name>
    </method>
  </method-permission> ...
<assembly-descriptor>
```

Middleware: Application Server - Architecture

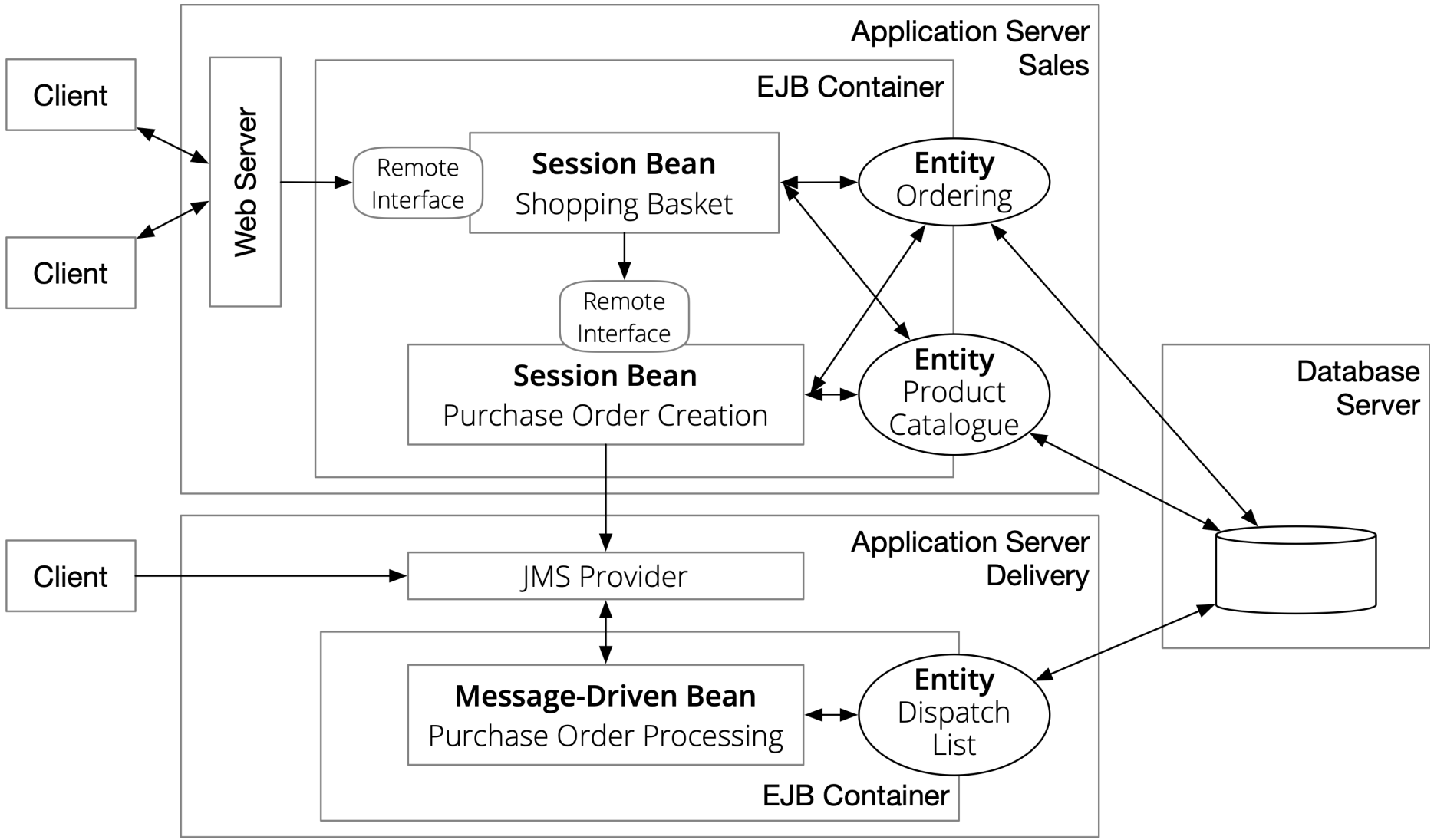
Product category for the “middle tier”

- Support for at least one component model
- Transaction support
- Security mechanisms
- Naming and Directory
- High scalability (multithreading, load balancing, replication and clustering)
- Standard support (communication, security, database integration, enterprise application integration, ...
- Integrated development environment with rich tool support

Examples:

- Java EE: Oracle J2EE, IBM WebSphere Application Server, and many others
- Microsoft .NET
- Open Source: Apache TomEE etc.

Online Shop with Enterprise JavaBeans



Advantages and Drawbacks

Advantages

- Separation of code and configuration
- Simplified programming interfaces for using of system services
- Specific tools for component-based software development
- Component model and platform:
  Enterprise JavaBeans, OSGi and Microsoft .NET-Components

Drawbacks

- Often referred to as Monolith
- Mainly use of homogeneous component model and platform
- Often bound to one programming language
- Close coupling of components within intranets of companies
- Limited flexibility for changing business processes
- Limitations for Scalability

SERVICE ORIENTED ARCHITECTURES AND MICROSERVICES ARCHITECTURE

What is a service?

*A **service** is a mechanism to enable access to one or more capabilities, where the access is provided using a prescribed interface and is exercised consistent with constraints and policies as specified by the service description.*

OASIS reference model

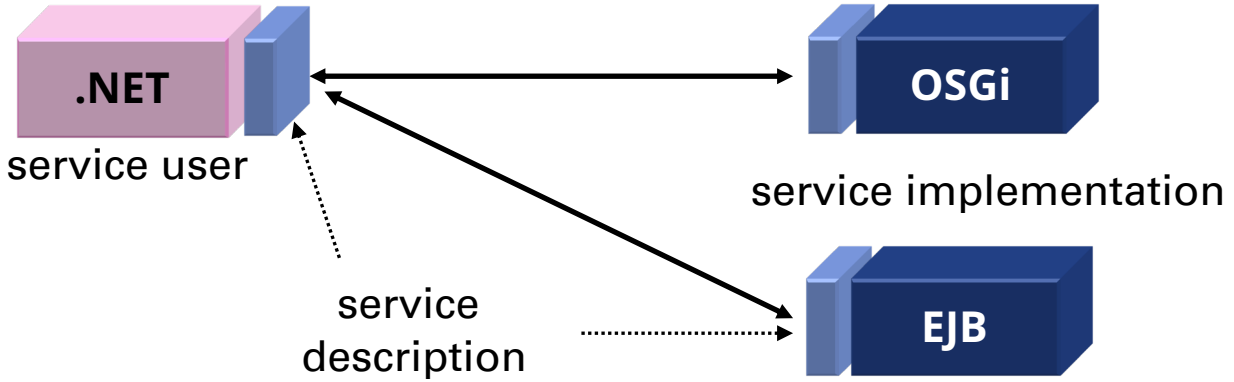
Enabling of access to capabilities (functionality with link to real world effect)

Service description defining constraints and policies

Functionality provided by formally specified interface

- Encapsulated functionality
- Implementation transparency

Loose coupling



Microservices architecture

designing software applications as suites of independently deployable services

J. Lewis and M. Fowler, https://www.martinfowler.com/microservices/

*In short, the **microservice architectural style** is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.*

J. Lewis and M. Fowler, https://www.martinfowler.com/microservices/

Microservices Architecture – Characteristics

https://www.martinfowler.com/articles/microservices.html

Single Responsibility/Single Concern:

- Each microservice is responsible for a specific business function or capability and performs that function independently
- Does one thing and one thing only
- Developed and operated as product by one team – „you build, you run it“

Componentization via Services

- Well-encapsulated
- Carry-its-own-data – accessible only via the service interface
- Transportable unit for deployment and replication

Composition

- Application assembled as suite of services
- Smart endpoints and dump pipes (as decoupled and as cohesive as possible)
- Infrastructure Automation

Task: Microservices Characteristics

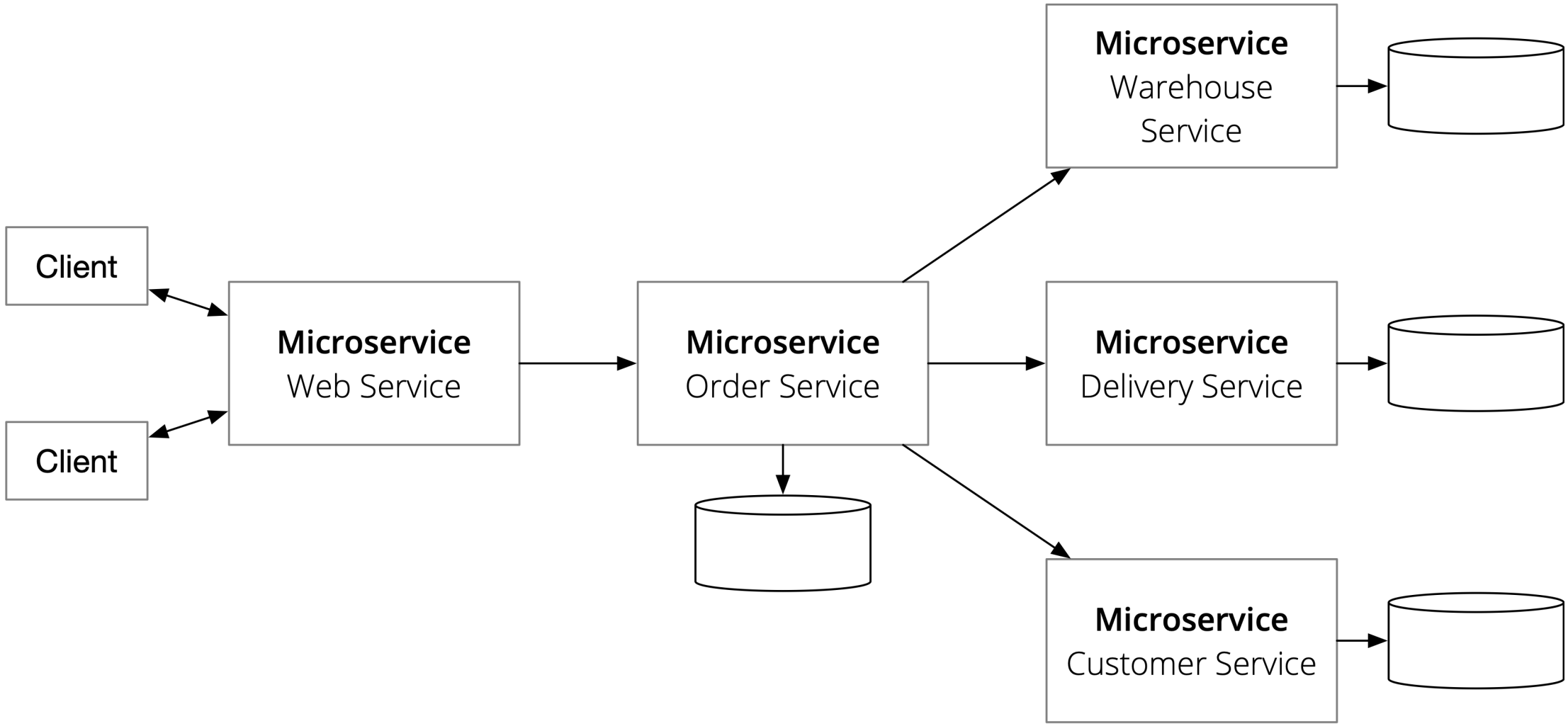
An online shop for selling books should allow users to browse in the product catalogue so see available books with prices and description as well as ratings given by other users.

Task: Sketch a microservices architecture of the online book shop. Use a simple "lines and boxes" notation and choose an appropriate level of detail to sketch a high level architecture of the complete system setup. Give rational, why you have broken down the system like you did.

- Think about the relation of coupling and cohesion at one hand and the Microservices characteristics of „carry it‘s own data“ and „Does one thing and one thing only“ at the other hand.
- Compare your approach with a Three-tier architecture, Which tiers are related to a Microservice?

Upload your sketch to the “Upload“ folder in OPAL

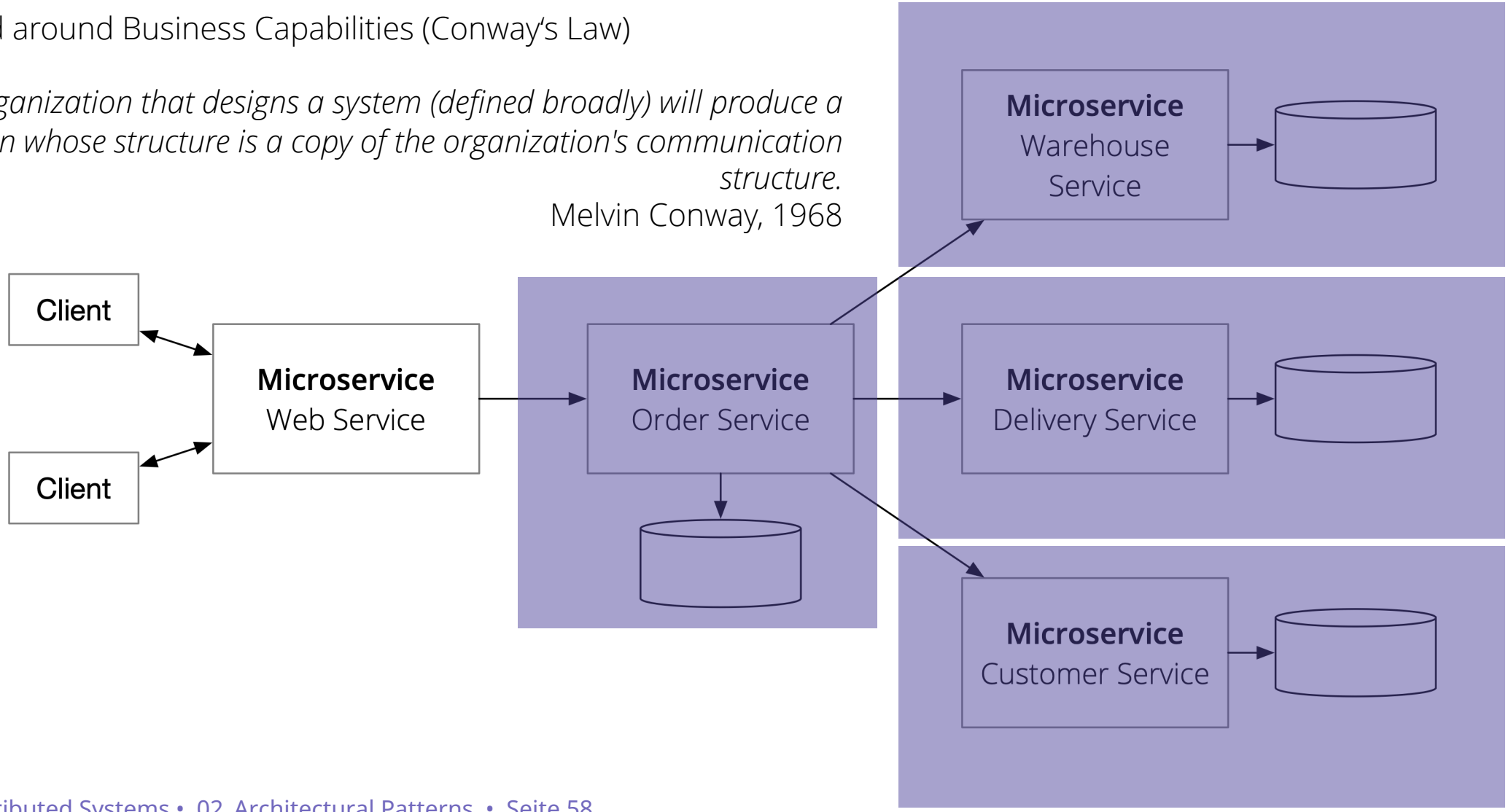
Online Shop - Microservices



Online Shop - Microservices

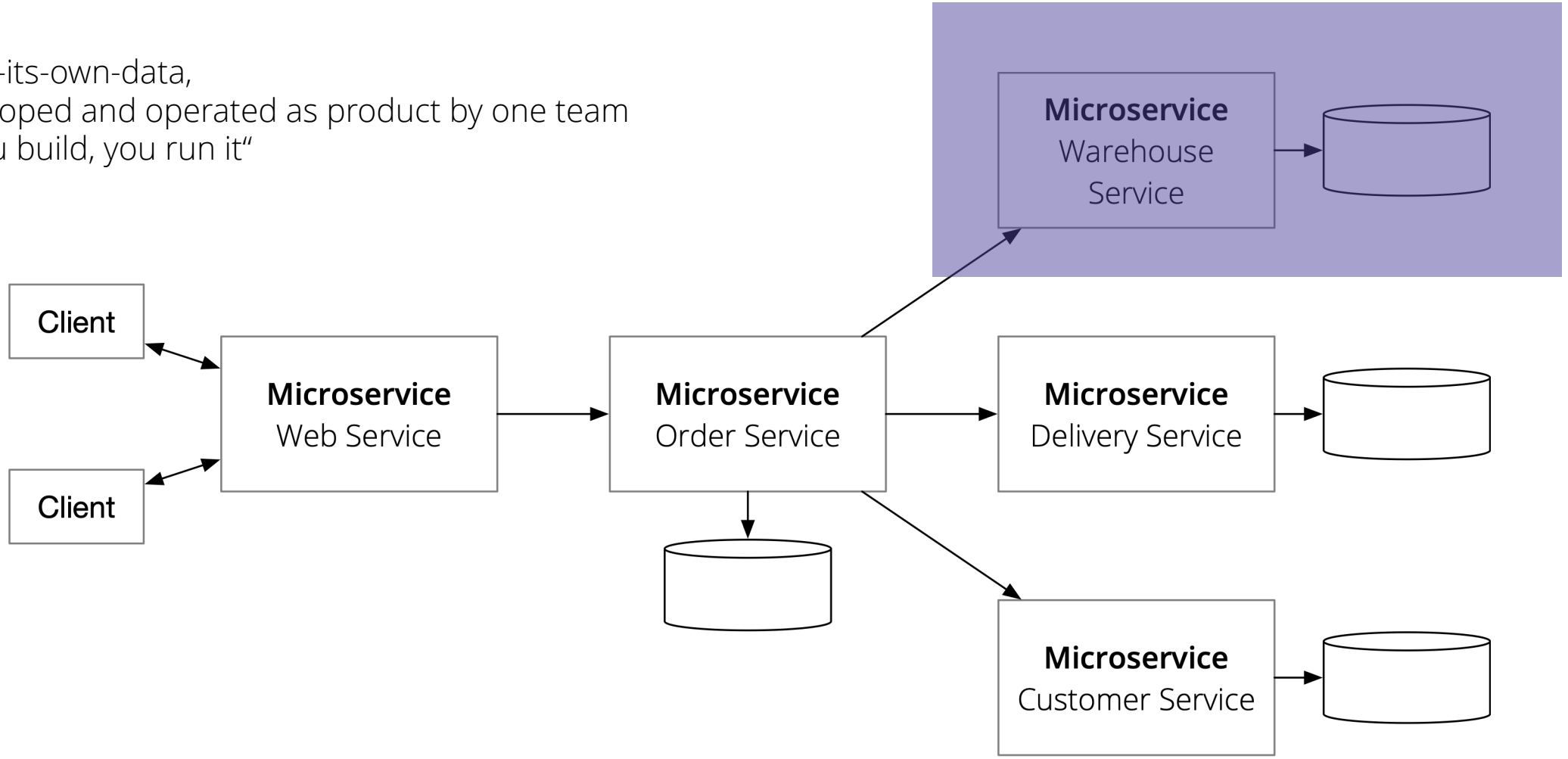
Organized around Business Capabilities (Conway's Law)

Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure.
Melvin Conway, 1968



Online Shop – Microservices

Carry-its-own-data,
Developed and operated as product by one team
– „you build, you run it“



Orchestration and Choreography

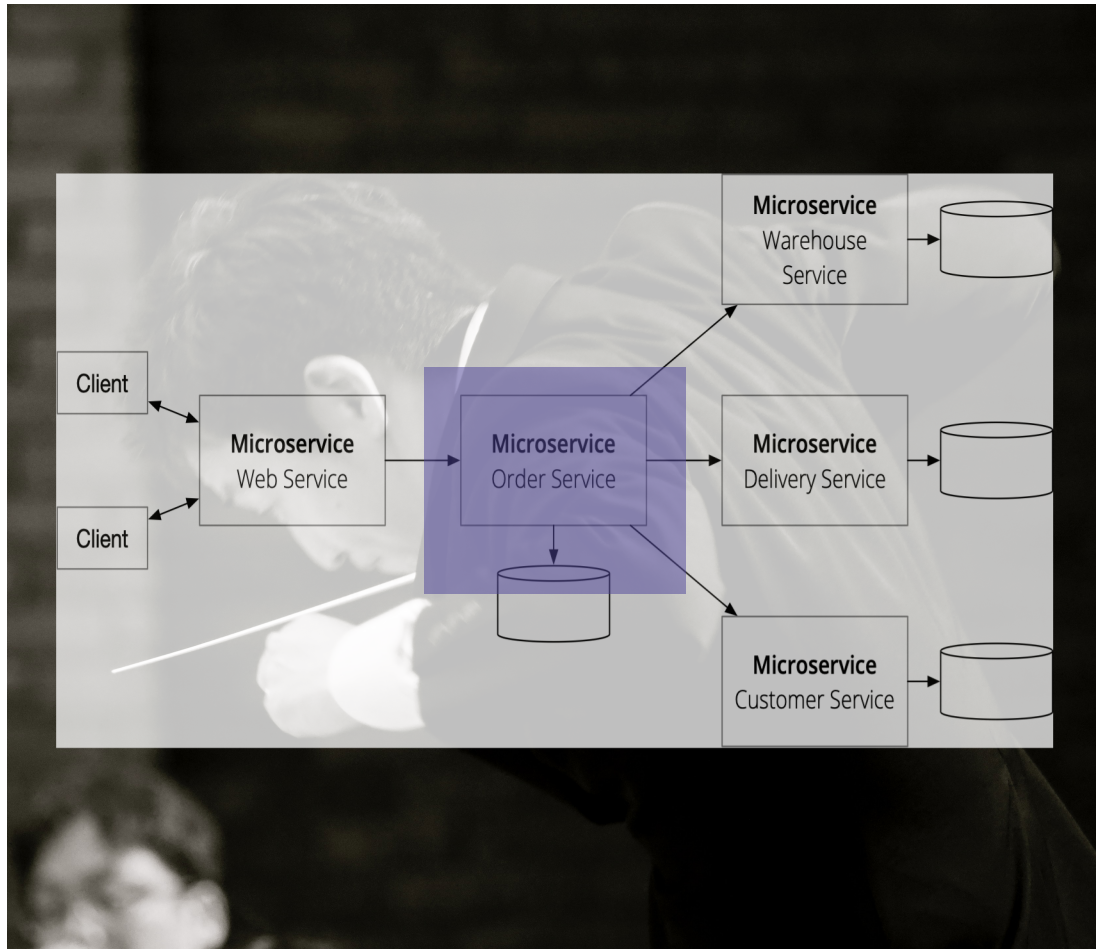


https://unsplash.com/de/fotos/mann-in-schwarzem-langarmhemd-halt-madchen-in-schwarzem-langarmhemd-zeKGi6UZ93w



https://unsplash.com/de/fotos/graustufenfoto-eines-vogelschwarms-sbpZBs1qR9k

Orchestration and Choreography



https://unsplash.com/de/fotos/mann-in-schwarzem-langarmhemd-halt-madchen-in-schwarzem-langarmhemd-zeKGi6UZ93w



https://unsplash.com/de/fotos/graustufenfoto-eines-vogelschwarms-sbpZBs1qR9k

Decentralized Systems

*A **decentralized system** is one which requires multiple parties to make their own independent decisions. In such a decentralized system, there is no single centralized authority that makes decisions on behalf of all the parties. Instead each party, also called a peer, makes local autonomous decisions towards its individual goals which may possibly conflict with those of other peers. Peers directly interact with each other and share information or provide service to other peers.*

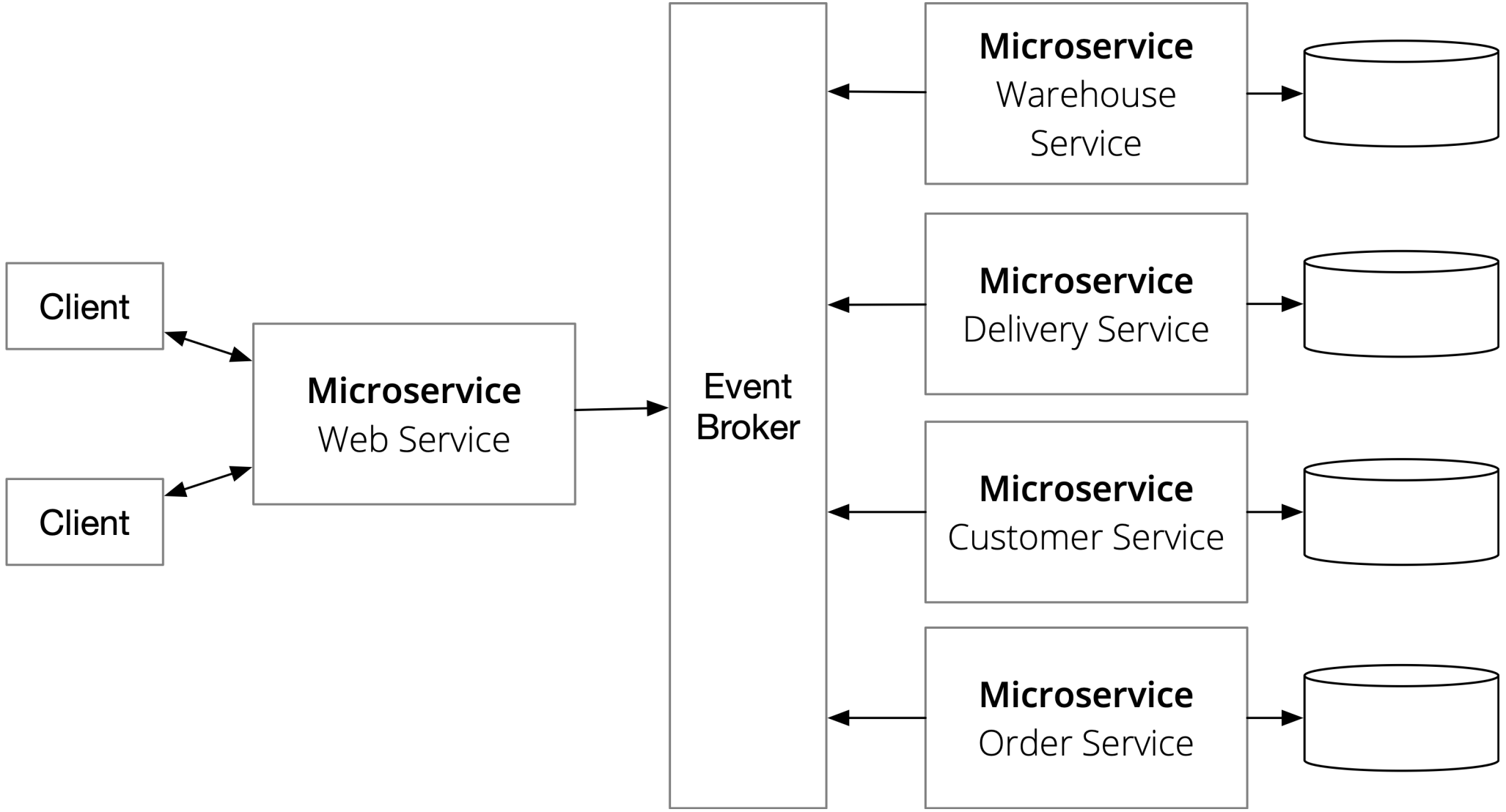
[Rohit Khare]

- Needs a number of autonomous entities
- Requires coordination
- Not a synonym for distributed
- Opposite is centralized

Example: ant colony

- all ants act upon local information and local interactions to collectively create complex, global behaviour.

Online Shop - Choreography



Event-Driven Architecture

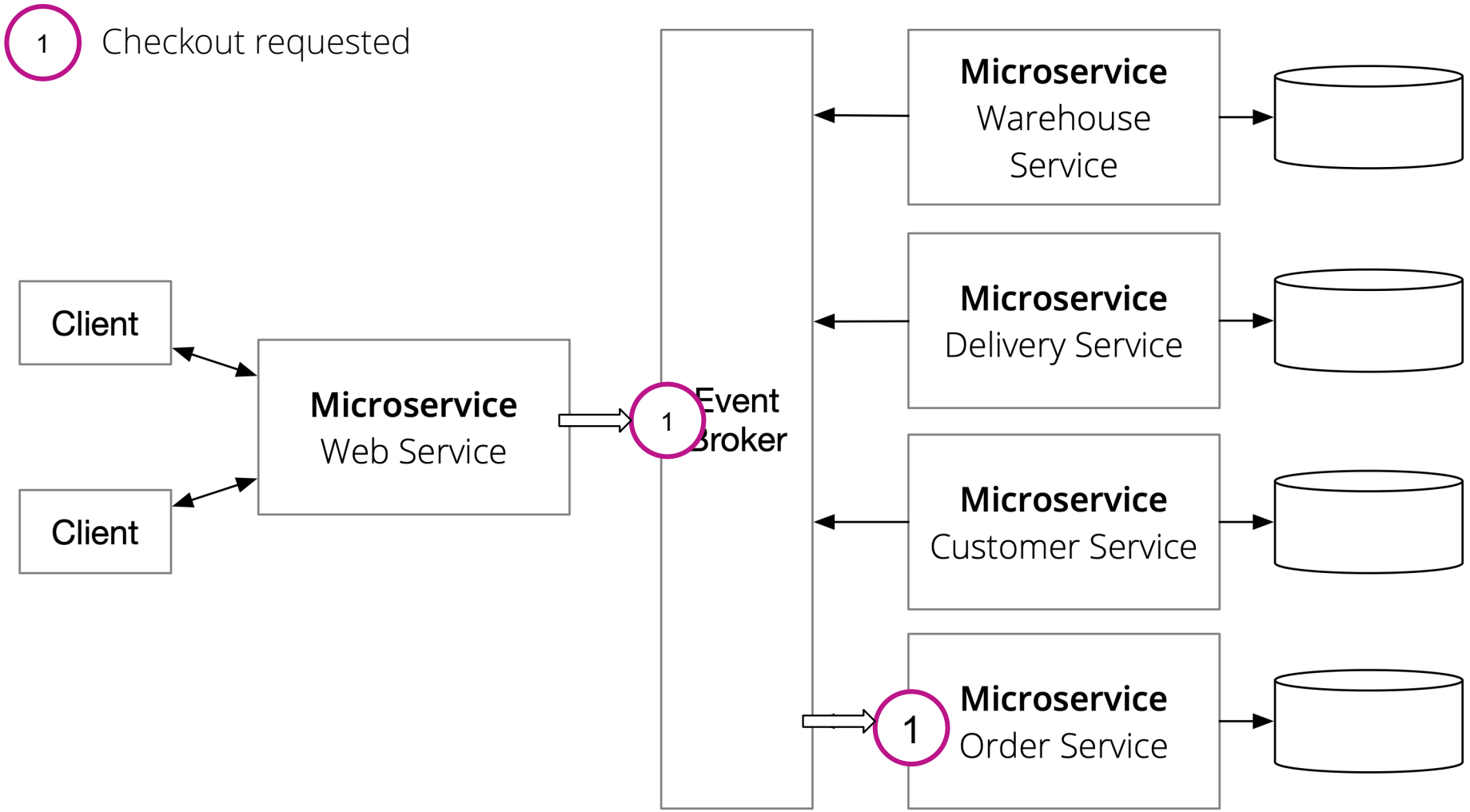
Event-Driven Systems react to events from the outside world and (other) system components
(Martin Fowler, “Focusing on Events”, martinfowler.com)

- Event: signals that something has happened or changed
- Event message: data structure representing the event containing
  - Source data: describing the event (immutable)
  - Processing data: describing processing performed on the event by an application system

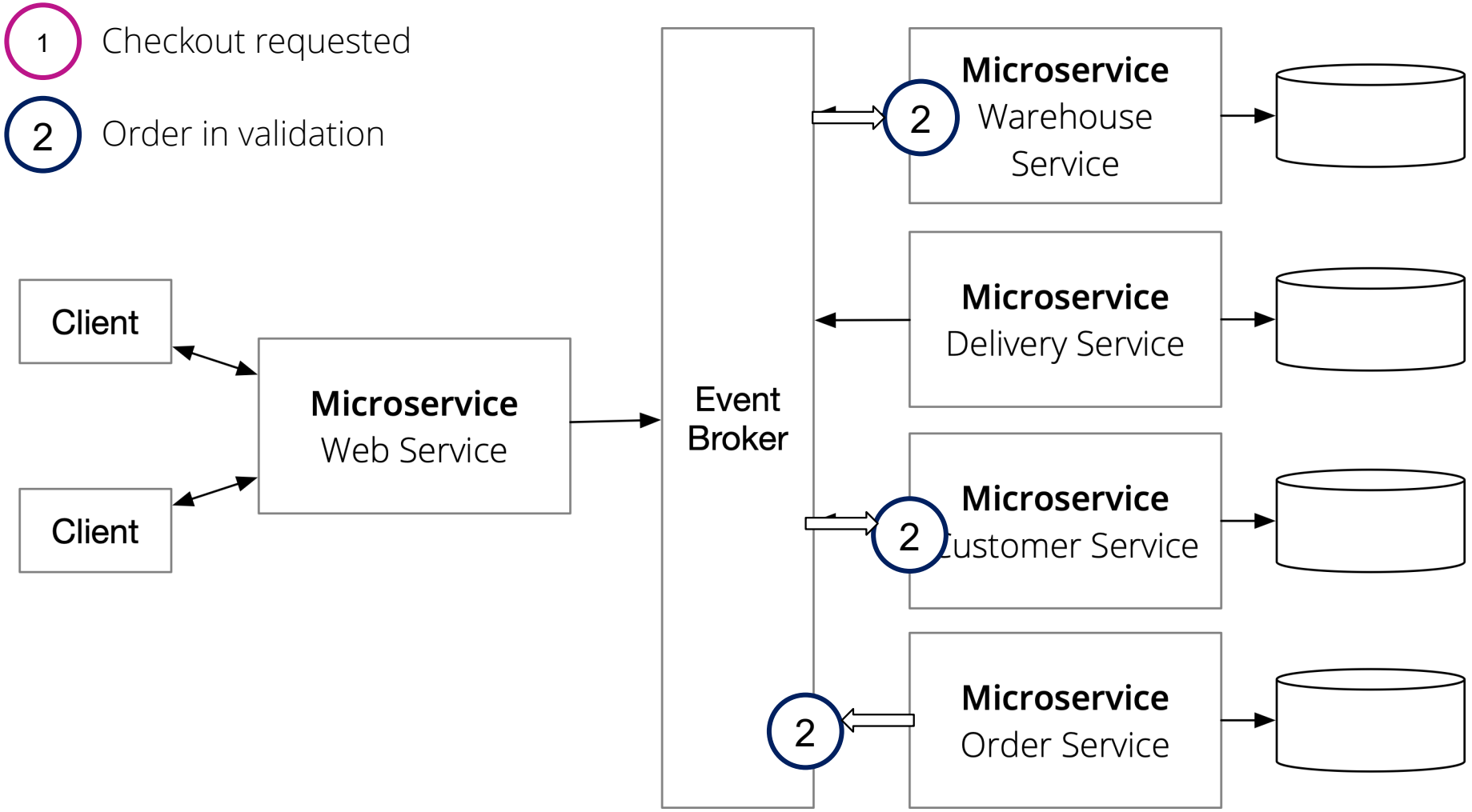
- System components are decoupled based on event messages (publish/subscribe)
  - Identity: Messages do not carry receiver information
  - Time: Messages can be queued and forwarded

- Support for decentralized systems since events are interpreted by receiver

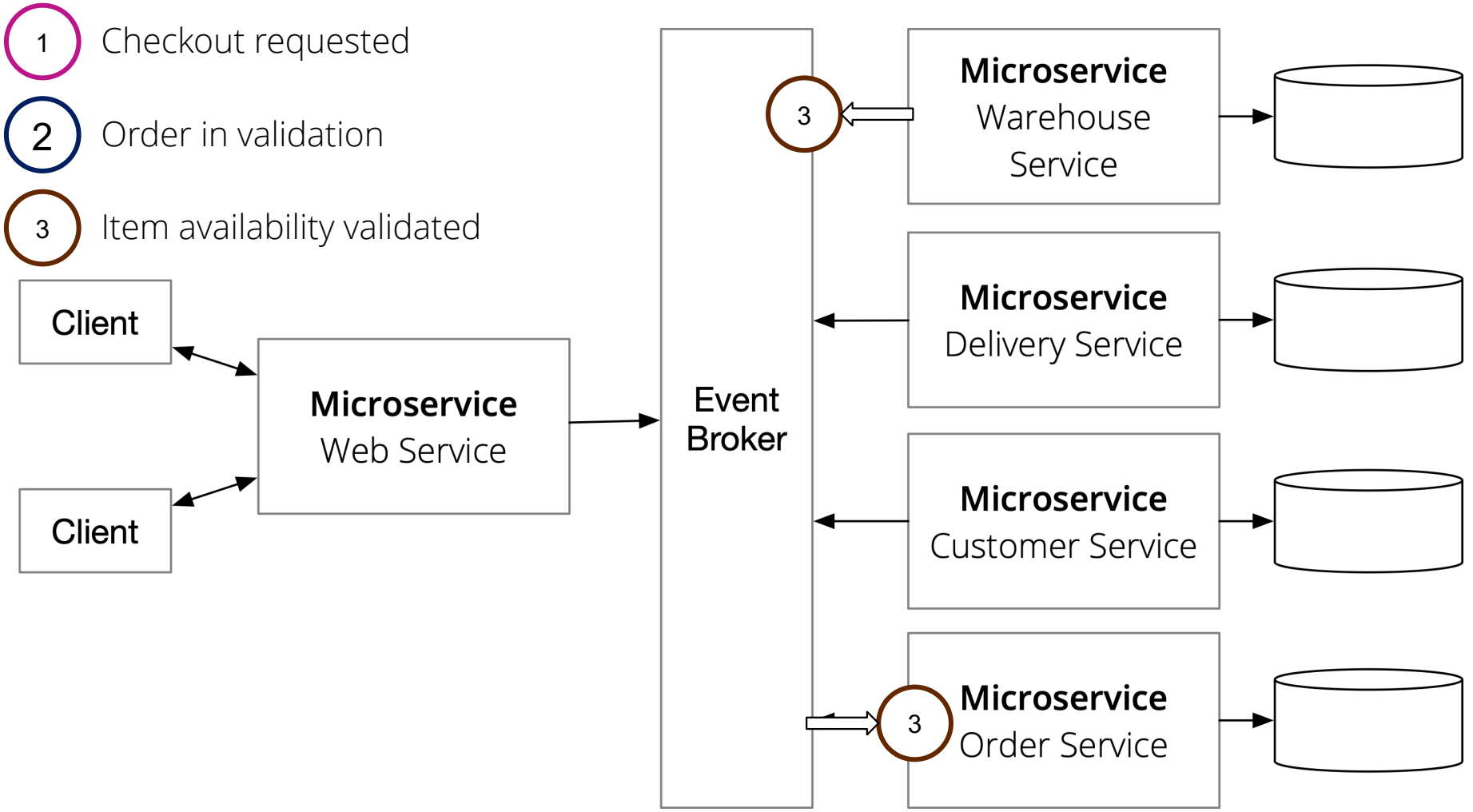
Online Shop - Choreography



Online Shop - Choreography



Online Shop - Choreography



Task: Book Shop with Choreographed Microservice Architecture

An online shop for selling books should allow users to browse in the product catalogue so see available books with prices and description as well as ratings given by other users.

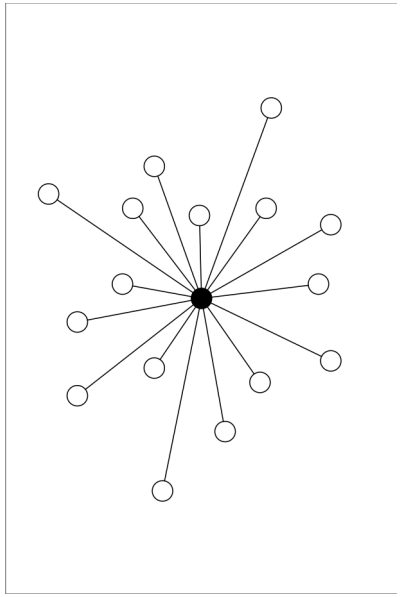
Task: Sketch a choreographed microservices architecture of the online book shop. Use a simple "lines and boxes" notation and choose an appropriate level of detail to sketch a high level architecture of the complete system setup. Give rational, why you have broken down the system like you did.

Peer-to-Peer and REST

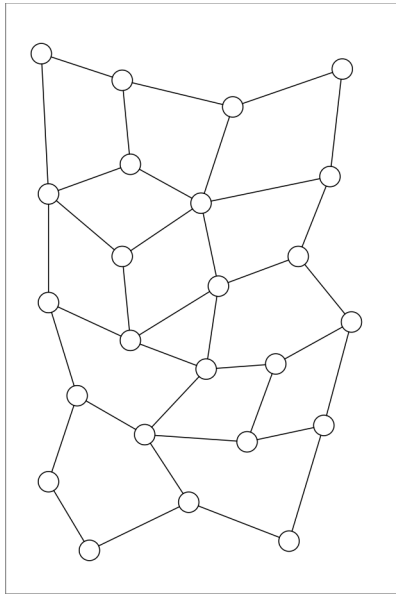
ALTERNATIVE ARCHITECTURAL STYLES

Peer-to-Peer Architectures

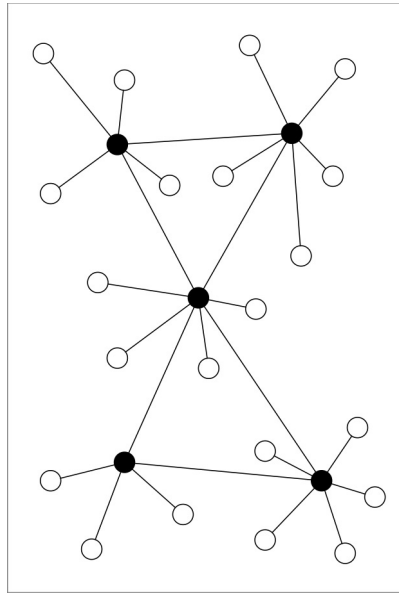
Centralized



Decentralized



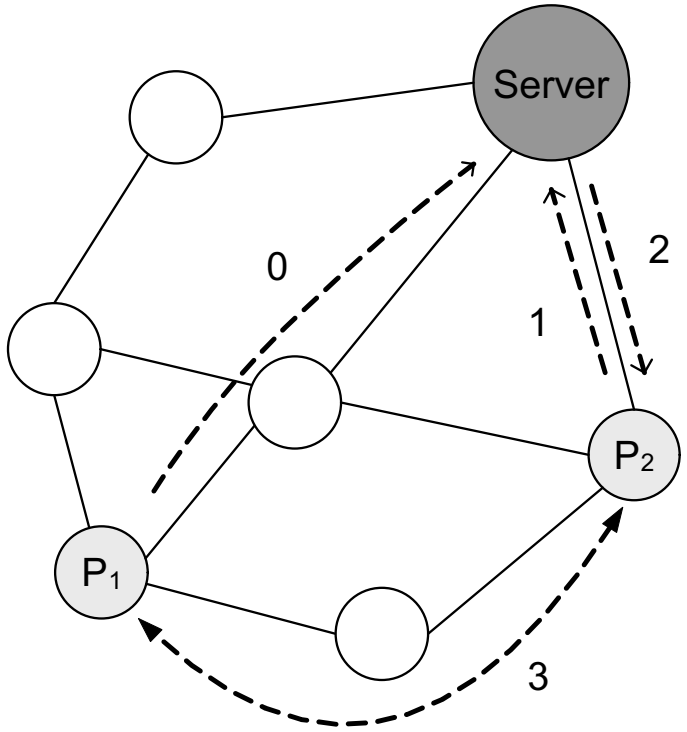
Hierarchical



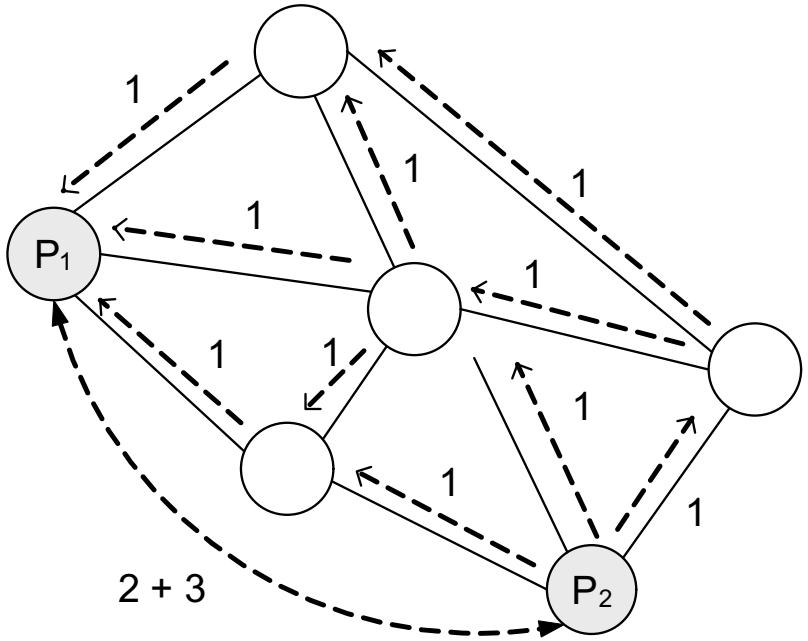
- Decentralized: each node is a peer acting as provider and user of services and data
  - No single node containing all system functionality or data
  - Mechanism for discovering services and data necessary
  - Several architectures are possible (centralized, decentralized, hierarchical, structured)
- Application areas:
  - File Sharing, Collaboration

Service/Data Discovery and Access in P2P Systems

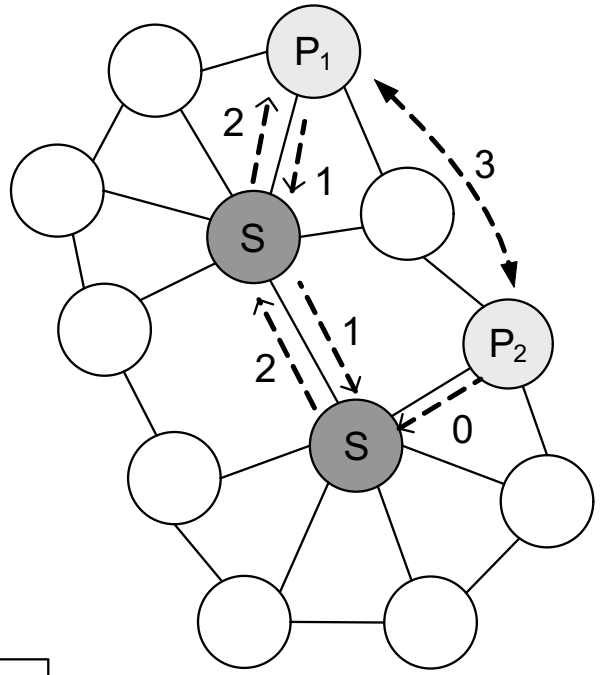
Centralized



Decentralized



Hierarchically



0: register, 1: search, 2: acquaint, 3: use

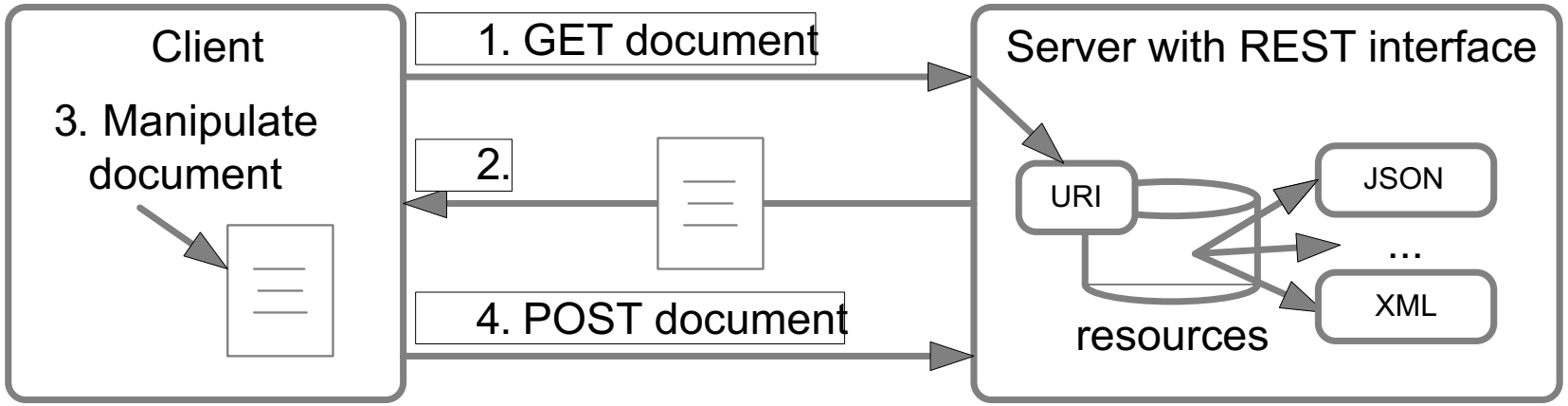
Representational StateTransfer - REST

Architectural style introduced by Roy Fielding (2000)

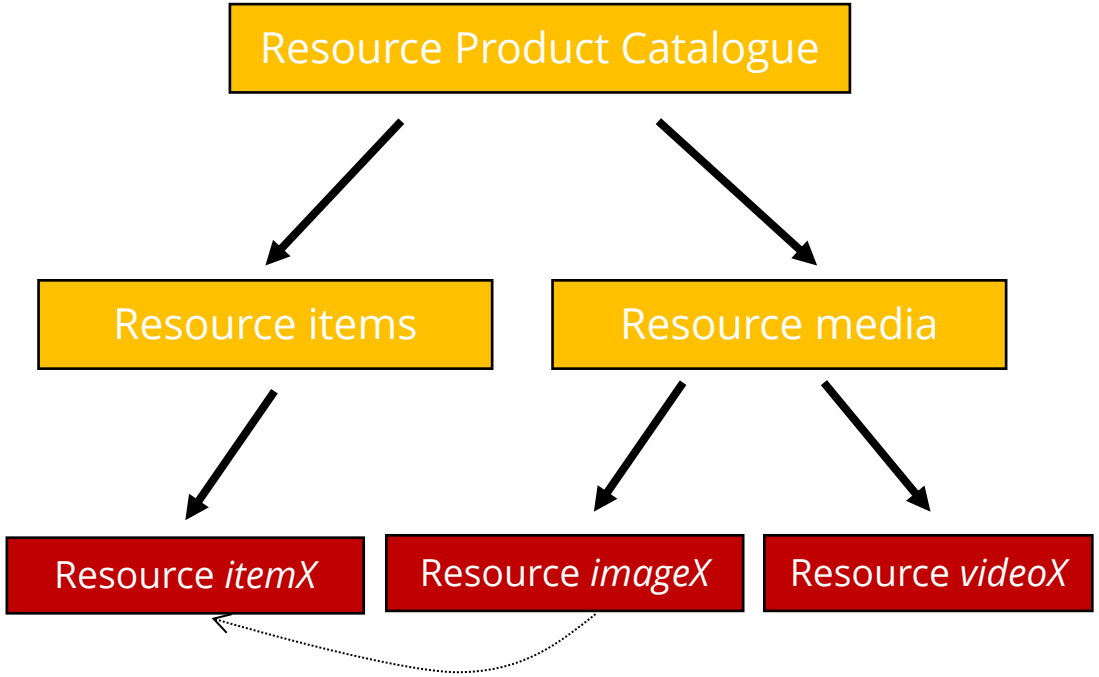
- Broadly adapted architecture style in the web
- Based on a client/server architecture

Servers provide access to resources instead of interfaces to operations

- **Resource:** web page, data collection, image, ... (e.g., product catalogue, customer data)
- Every resource has to be available through a **unique identifier** (URI)
- Fixed set of operations on resources: **C**reate, **R**ead, **U**pdate, **D**elete – **CRUD**
- **CRUD operations** usually mapped to **HTTP methods**



REST - URIs and Methods



http://myserver.com/productcatalogue

http://myserver.com/productcatalogue/items

http://myserver.com/productcatalogue/items/itemX

Resource type	GET	POST	PUT	DELETE
collection http://myserver.com/productcatalog/items	**List** URIs and details of resources in collection	**Create** new resource in the collection	**(Update/Replace** every resource in the collection)	**(Delete** addressed collection)
item http://myserver.com/productcatalogue/items/itemX	**List** details of addressed resource in appropriate format	-	**Update** an existing resource	**Delete** addressed resource